Laporan Tugas Besar 1
IF3170 Inteligensi Artifisial



Disusun oleh:

13524026 Made Branenda Jordhy
13524031 Vincent Rionarlie
13524067 Bryan Pratama Putra Hendra

Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
Semester I 2026/2027

DAFTAR ISI

DAFTAR ISI **0**
DESKRIPSI PERSOALAN **3**
I. Deskripsi Masalah 3
II. Representasi State 4
III. Fungsi Objektif dan Batasan Masalah 5
PEMBAHASAN **7**
I. Implementasi Algoritma Local Search 7
A. Hill Climbing dan Variannya 7
a. Steepest Ascent Hill Climbing 7
b. Sideways Move Hill Climbing 11
c. Stochastic Hill Climbing 13
d. Random Restart Hill Climbing 16
B. Simulated Annealing 18
C. Genetic Algorithm 21
II. Rancangan Eksperimen 26
A. Tujuan Eksperimen 26
B. Testcase Utama Eksperimen 26
C. Testcase Bonus Multi-Truk 27
D. Konfigurasi Eksperimen 28
E. Hasil Eksperimen Setiap Algoritma 28
1. Hill Climbing 28
2. Simulated Annealing 38
3. Genetic Algorithm 41
F. Analisis Pengaruh Parameter 50
1. Hill Climbing dan variannya 50
2. Simulated Annealing 51
3. Genetic Algorithm 52
G. Perbandingan Kinerja Antar Algoritma 52
III. Pengembangan Fitur Bonus 55
KESIMPULAN & SARAN **57**
I. Kesimpulan 57
II. Saran 57
LAMPIRAN **58**
I. Link Repository 58
II. Pembagian Tugas 58
III. Referensi 58
IV. Statement Penggunaan AI 59

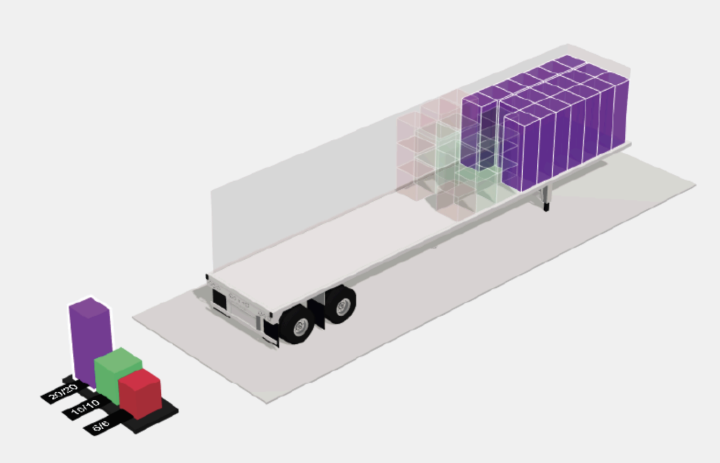
DESKRIPSI PERSOALAN

I. Deskripsi Masalah

Persoalan yang dibahas dalam tugas ini adalah optimasi pemilihan dan penataan paket di dalam ruang muat truk berbentuk balok. Terdapat sejumlah paket dengan ukuran, berat, nilai, dan sifat kerapuhan yang berbeda. Setiap paket dapat dimasukkan ke dalam truk atau ditinggalkan di luar apabila tidak memungkinkan untuk dimuat.

Tujuan optimasi adalah memaksimalkan total nilai paket yang berhasil dimuat dengan tetap memenuhi seluruh batasan penempatan. Batasan tersebut meliputi dimensi ruang muat, kapasitas berat truk, larangan tumpang tindih antarpaket, serta ketentuan penopangan dan penumpukan paket rapuh. Pemilihan paket dan penentuan susunannya merupakan dua keputusan yang saling berkaitan. Sekumpulan paket yang memenuhi kapasitas berat belum tentu dapat disusun di dalam truk karena keterbatasan ruang dan bentuk paket. Sebaliknya, susunan yang memanfaatkan ruang dengan baik belum tentu menghasilkan total nilai terbesar.

Persoalan ini dimodelkan sebagai optimasi penataan balok tiga dimensi. Paket dapat dirotasi dengan menukar orientasi dimensinya, tetapi setiap sisinya tetap sejajar dengan sumbu ruang muat. Penyelesaian dilakukan menggunakan Hill Climbing beserta variannya, Simulated Annealing, dan Genetic Algorithm. Algoritma tersebut digunakan untuk mencari susunan bernilai tinggi dalam ruang pencarian yang terdiri atas berbagai pilihan paket, posisi, dan orientasi.



Gambar 1. Ilustrasi penataan paket dengan ukuran berbeda dalam ruang muat tiga dimensi (source: https://github.com/chadiik/freight-packer)

II. Representasi State

Truk direpresentasikan sebagai ruang tiga dimensi dengan lebar W, panjang L, tinggi H, dan kapasitas berat maksimum C. Sumbu x, y, dan z masing-masing menyatakan arah lebar, panjang, dan tinggi. Titik asal $(0, 0, 0)$ berada pada salah satu sudut lantai ruang muat.

Setiap paket i memiliki identitas unik, dimensi awal (w_i, l_i, h_i), nilai v_i, berat m_i, serta atribut yang menyatakan apakah paket bersifat rapuh (*fragile*).

Sebuah *state* menyatakan kondisi penempatan seluruh paket. Untuk setiap paket, informasi yang disimpan berupa posisi dan orientasi apabila paket berada di dalam truk, atau penanda bahwa paket berada di luar truk. Secara matematis, representasi tersebut dapat dituliskan sebagai berikut.

$$s = \{i \mapsto p_i \mid i = 1, 2, \ldots, n\} p_i = \begin{cases} (x_i, y_i, z_i, o_i), & \text{jika paket } i \text{ dimuat}, \\ \varnothing, & \text{jika paket } i \text{ tidak dimuat}. \end{cases}$$

Koordinat (x_i, y_i, z_i) menunjukkan sudut paket dengan koordinat terkecil pada setiap sumbu dan menggunakan bilangan bulat. Sementara itu, o_i merupakan permutasi dimensi paket yang menentukan orientasinya. Terdapat enam kemungkinan permutasi, meskipun beberapa di antaranya menghasilkan bentuk yang sama apabila paket memiliki sisi dengan panjang yang sama. Dalam implementasi, paket yang tidak dimuat direpresentasikan dengan nilai None.

Perubahan dari satu *state* ke *state* tetangga dapat dilakukan melalui tiga operasi utama:

1. **Pertukaran (*swap*)**, yaitu menukar posisi dua paket, termasuk pertukaran antara paket di dalam dan di luar truk.
2. **Pemindahan (*relocate*)**, yaitu mengubah posisi paket, memasukkan paket ke dalam truk, atau mengeluarkannya.
3. **Rotasi (*rotate*)**, yaitu mengubah orientasi paket melalui rotasi 90 derajat terhadap salah satu sumbu.

Representasi ini memungkinkan proses pencarian mengevaluasi pemilihan paket sekaligus susunan geometrisnya

III. Fungsi Objektif dan Batasan Masalah

A. Fungsi Objektif

Kualitas solusi ditentukan oleh total nilai paket yang dimuat. Misalkan a_i adalah indikator bernilai 1 apabila paket i berada di dalam truk dan 0 apabila berada di luar truk. Fungsi objektif dirumuskan sebagai berikut:

$$\max_{s \in \mathcal{F}} f(s) = \sum_{i=1}^{n} v_i a_i \qquad a_i = \begin{cases} 1, & \text{jika paket } i \text{ dimuat,} \\ 0, & \text{jika paket } i \text{ tidak dimuat.} \end{cases}$$

Himpunan F merupakan seluruh *state* yang memenuhi batasan masalah. Dengan demikian, solusi yang dicari adalah susunan valid dengan total nilai paket terbesar. Jumlah paket dan pemanfaatan volume dapat membantu menjelaskan hasil penataan, tetapi keduanya bukan fungsi objektif utama dalam program.

B. Batasan Masalah

- Paket berada di dalam batas ruang muat

  Seluruh bagian paket yang dimuat harus berada di dalam truk. Jika dimensi paket setelah dirotasi adalah (d_{xi}, d_{yi}, d_{zi}), maka berlaku:

$$\begin{aligned} 0 \le x_i, \qquad & x_i + d_{xi} \le W, \\ 0 \le y_i, \qquad & y_i + d_{yi} \le L, \\ 0 \le z_i, \qquad & z_i + d_{zi} \le H. \end{aligned}$$

- Paket tidak saling tumpang tindih

  Dua paket tidak boleh menempati volume yang sama. Untuk setiap pasangan paket yang dimuat, keduanya harus terpisah pada sedikitnya satu sumbu. Paket diperbolehkan bersentuhan pada sisi, rusuk, atau titik sudut.

- Total berat tidak melebihi kapasitas truk

  Jumlah berat seluruh paket yang dimuat tidak boleh melampaui kapasitas maksimum truk:,

$$\sum_{i=1}^{n} m_i a_i \leq C.$$

- Paket memiliki penopang

  Setiap paket harus berada di atas lantai truk atau ditopang oleh paket lain. Paket dengan $z_i = 0$ dianggap ditopang langsung oleh lantai. Untuk paket di atas lantai, permukaan atas paket penopang harus berada tepat pada ketinggian alas paket tersebut dan memiliki irisan area alas yang positif. Dalam model yang digunakan, seluruh alas paket tidak harus tertopang.

- Paket rapuh tidak boleh menjadi penopang

  Paket dengan atribut *fragile* tidak diperbolehkan menopang paket lain. Paket rapuh tetap dapat dimuat di lantai maupun di atas paket nonrapuh selama memenuhi batasan lainnya.

- Representasi penempatan valid

  Setiap paket harus tercatat tepat satu kali dalam *state*, baik sebagai paket yang dimuat maupun yang berada di luar truk. Paket yang dimuat harus memiliki koordinat bilangan bulat dan orientasi yang sesuai dengan permutasi dimensi yang diperbolehkan.

  Dalam pencarian Hill Climbing dan Simulated Annealing, program juga menggunakan skor berpenalti untuk mengevaluasi kandidat yang masih melanggar batasan sebagai berikut.

  $$F(s) = f(s) - \lambda V(s).$$

  Pada persamaan tersebut, $V(s)$ menyatakan ukuran pelanggaran yang dihitung dari jumlah paket di luar batas, volume tumpang tindih, pelanggaran penopangan atau penumpukan paket rapuh, serta kelebihan berat. Parameter λmerupakan bobot penalti yang secara bawaan ditetapkan sebesar jumlah nilai seluruh paket. Skor ini membantu mengarahkan pencarian menuju susunan yang memenuhi batasan. Kelayakan solusi akhir tetap diperiksa secara terpisah dari total nilai paketnya.

PEMBAHASAN

I. Implementasi Algoritma Local Search

A. Hill Climbing dan Variannya

a. Steepest Ascent Hill Climbing

Hill climbing merupakan algoritma paling umum dalam implementasi local search yang memperoleh solusi susunan paket dengan mencari optimum lokal yang akan dibandingkan satu sama lain untuk diambil optimum global-nya.

Algoritma hill climbing yang paling umum diketahui adalah Steepest Ascent Hill Climbing. Varian algoritma tersebut merupakan algoritma utama pada tugas besar kami. Pada varian ini, hill climbing akan mencari optimum lokal dengan memilih tetangga langsung yang lebih baik daripada titik saat itu. Sayangnya, algoritma tersebut rawan terjebak dalam optimum lokal karena algoritma tidak akan mengambil tetangga yang lebih buruk darinya.

1. Inisialisasi state awal

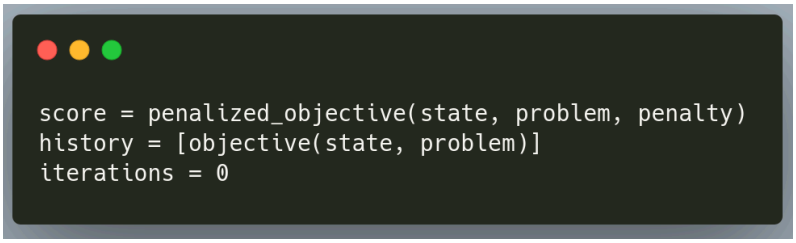
   Algoritma dimulai dengan pembangkitan `random_state()` dengan p_{insert} = 0.5 untuk memperoleh state awal. Paket diproses dalam urutan yang acak, dengan setiap paket berpeluang 50% dimasukkan ke salah satu titik ekstrim pada saat itu dengan orientasi yang acak. Jika peletakan paket tersebut ternyata tidak muat di truk, paket dibiarkan menjadi `None`.

```python
def random_state(problem: Problem, rng: random.Random, p_insert: float = 0.5) -> State:
    state: State = {pid: None for pid in problem.packages}
    order = list(problem.packages)
    rng.shuffle(order)

    for pid in order:
        if rng.random() >= p_insert:
            continue
        point = rng.choice(extreme_points(state, problem))
        placement = (point[0], point[1], point[2], rng.choice(ORIENTATIONS))
        if _fits(problem, pid, placement):
            state[pid] = placement
    return state
```

2. Inisialisasi skor dan history

`_steepest_climb` menghitung `score` = g(s_0) sebagai baseline perbandingan, lalu mencatat `history` = [f(s_0)]. Ada dua nilai yang berbeda. Pertama, skor yang dipakai untuk dibandingkan untuk pengambilan keputusan adalah g, sedangkan history nilai yang telah muncul adalah f. Pemisahan ini disengaja agar `history` menyimpan nilai yang belum diubah pada g.



```
score = penalized_objective(state, problem, penalty)
history = [objective(state, problem)]
iterations = 0
```

Karena history menyimpan f, history tidak harus monoton naik. Contohnya, jika state saat ini memasukkan paket yang overlap, paket tersebut akan dibuang. Akibatnya, f bisa turun.

Secara formal, history[0] = f(s_0). Setelah t langkah diterima, maka history[t] = f(s_t). Yang dijamin monoton adalah skor:

$$g(s_{t+1}) > g(s_t) + \varepsilon,$$

untuk setiap langkah yang diterima. Hubungan dari kedua fungsi f dan g adalah:

$$g(s_t) = f(s_t) - \lambda \cdot V(s_t)$$

3. Pencarian neighbor terbaik secara exhaustive

   Inti dari steepest ascent ada di loop for candidate in `all_neighbors(...)`. Setiap iterasi mengevaluasi seluruh neighbor yang valid secara batasan truk, menghitung nilai g masing-masing, lalu menyimpan kandidat yang skornya lebih baik dari skor terbaik sementara. Baseline `best_candidate_score = score` memastikan neighbor dengan skor yang sama tidak pernah dipilih.

```
while iterations < max_iter:
    best_neighbor = None
    best_candidate_score = score
    for candidate in all_neighbors(state, problem):
        candidate_score = penalized_objective(candidate, problem, penalty)
        if candidate_score > best_candidate_score + EPSILON:
            best_candidate_score = candidate_score
            best_neighbor = candidate
```

`all_neighbors()` menghasilkan tiga move secara berurutan, yaitu swap, relocate, dan rotate, dimana urutan evaluasi deterministik. Jika ada dua neighbor dengan skor yang sama dan sama-sama lebih baik, neighbor yang muncul lebih dulu yang menang. Hal ini membuat steepest ascent sangat bergantung pada urutan enumerasi.

```
def all_neighbors(state: State, problem: Problem) -> Iterator[State]:
    ids = list(state)

    for i, a in enumerate(ids):
        for b in ids[i + 1:]:
            neighbor = swap(state, problem, a, b)
            if neighbor is not None:
                yield neighbor

    for pid in ids:
        for target in [*extreme_points(state, problem, exclude=pid), OUTSIDE]:
            neighbor = relocate(state, problem, pid, target)
            if neighbor is not None:
                yield neighbor

    for pid in ids:
        for axis in AXES:
            neighbor = rotate(state, problem, pid, axis)
            if neighbor is not None:
                yield neighbor
```

Secara matematis, satu iterasi penuh memiliki biaya:

$$O(|N(s)| \cdot n^2),$$

atau sekitar $O(n^4)$ pada worst case scenario. Karena alasan tersebut, `max_iter` dibuat relatif kecil meskipun state space-nya besar.

4. Kriteria berhenti

   Jika tidak ada satu pun neighbor yang melebihi nilai $score + \varepsilon$, `best_neighbor` tetap `None` dan loop berhenti. Kondisi ini

menandai optimum lokal terhadap $N(s)$. Jika ada kandidat neighbor, state berpindah ke kandidat tersebut, skor diperbarui, dan iterations bertambah. Loop juga berhenti saat iterations menyentuh `max_iter`, sehingga ada dua jalur terminasi, yaitu optimum lokal dan batas iterasi.

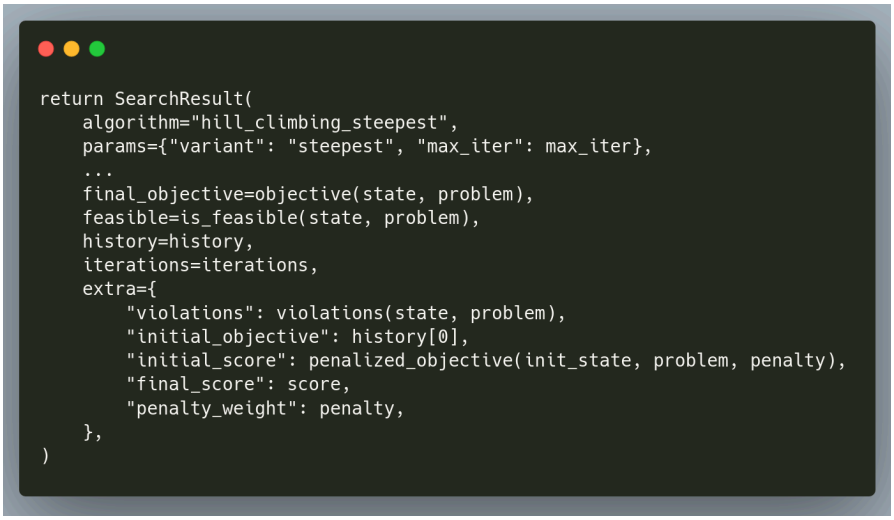
Hanya langkah yang benar-benar diterima yang akan dihitung sebagai iterasi. Hal ini membuat iterations bermakna sebagai panjang lintasan aktual, bukan jumlah evaluasi. Karena setiap langkah menaikkan skor minimal sebesar ε dan ruang state terbatas, steepest ascent pasti berhenti di optimum lokal jika `max_iter` tak hingga.

```
    if best_neighbor is None:
        break
    state = best_neighbor
    score = best_candidate_score
    iterations += 1
    history.append(objective(state, problem))

return state, history, iterations, score
```

5. Hasil pencarian

   Hasil akhir dibungkus dalam `SearchResult` dengan pemisahan antara `final_objective` (atau *f*) dan `final_score` (atau *g*). Selain itu, dimuat juga hasil berupa state awal, history, jumlah iterasi, feasibility, jumlah `violations`, objektif awal, skor awal, besaran penalti, dan durasi eksekusi. Feasibility dihitung melalui fungsi `is_feasible()`, yang memeriksa seluruh constraint secara eksplisit. `is_feasible()` memeriksa representasi state, batas, kapasitas, dan overlap.



```
return SearchResult(
    algorithm="hill_climbing_steepest",
    params={"variant": "steepest", "max_iter": max_iter},
    ...
    final_objective=objective(state, problem),
    feasible=is_feasible(state, problem),
    history=history,
    iterations=iterations,
    extra={
        "violations": violations(state, problem),
        "initial_objective": history[0],
        "initial_score": penalized_objective(init_state, problem, penalty),
        "final_score": score,
        "penalty_weight": penalty,
    },
)
```

b. Sideways Move Hill Climbing

Algoritma ini adalah modifikasi dari hill climbing biasa yang mengizinkan perpindahan ke tetangga dengan nilai yang sama dengan state saat ini, bukan hanya ke tetangga yang lebih baik. Hal ini berguna ketika pencarian berada di shoulder (dataran yang diikuti tanjakan) karena algoritma bisa terus bergerak sampai menemukan jalan naik.

Agar tidak tersangkut di optimum lokal datar, jumlah langkah menyamping dibatasi, misalnya maksimal 100 kali berturut-turut. Dengan batas ini, tingkat keberhasilan pencarian biasanya meningkat dibandingkan hill climbing tanpa langkah menyamping, dengan tambahan biaya langkah yang relatif kecil.

1. State tambahan untuk plateau

   Selain state, score, dan history, varian ini menyimpan `visited` berisi state-state pada plateau saat ini, dibekukan menjadi `frozenset` agar hashable dan bisa dimasukkan ke himpunan. Ada juga dua variabel: `sideways_moves` untuk total langkah datar sepanjang pencarian, dan `consecutive` untuk langkah berurutan.

```
state = dict(init_state)
score = penalized_objective(state, problem, penalty)
history = [objective(state, problem)]
iterations = 0
sideways_moves = 0
consecutive = 0
visited = {_freeze(state)}
```

`visited` adalah kunci untuk mencegah pencarian berputar-putar di plateau yang sama. Variabel ini adalah variabel utama yang memungkinkan sideways move HC untuk tidak bolak-balik antara dua state yang telah dikunjungi.

2. Mengumpulkan kandidat naik dan kandidat datar

   Dalam hal mencari neighbor, algoritma tetap mencari skor tertinggi seperti steepest, tetapi sekaligus mengumpulkan neighbor yang skornya sama dengan skor sekarang dan belum pernah dikunjungi pada plateau ini. Dua pengecekan ini dibangun dalam satu loop yang sama supaya akses ke neighbor yang sama hanya dilakukan sekali

   Pemisahan neighbor berdasarkan dua interval, yaitu $> g(s) + \varepsilon$ dan $|g(s') - g(s)| \leq \varepsilon$ membuat tiga kategori, yaitu lebih baik, datar, dan lebih buruk. Kategori lebih buruk diabaikan sepenuhnya dalam pengecekan. Pembanding untuk kategori datar adalah score saat ini, bukan `best_candidate_score`.

```
for candidate in all_neighbors(state, problem):
    candidate_score = penalized_objective(candidate, problem, penalty)
    if candidate_score > best_candidate_score + EPSILON:
        best_candidate_score = candidate_score
        best_neighbor = candidate
    elif (
        abs(candidate_score - score) <= EPSILON
        and _freeze(candidate) not in visited
    ):
        equal_neighbors.append(candidate)
```

3. Prioritas perbaikan

   Jika ada neighbor yang lebih baik, algoritma langsung pindah ke sana, mereset `consecutive` ke nol, dan mereset `visited`

hanya ke state baru. Jika tidak ada perbaikan, algoritma memeriksa dua kondisi berhenti, yaitu apakah `consecutive` sudah mencapai `max_sideways`, dan apakah masih ada kandidat datar yang belum dikunjungi. Bila keduanya lolos, satu kandidat datar dipilih secara acak dan dihitung sebagai langkah.

```
if best_neighbor is not None:
    state = best_neighbor
    score = best_candidate_score
    consecutive = 0
    visited = {_freeze(state)}
    iterations += 1
    history.append(objective(state, problem))
    continue

if consecutive >= max_sideways:
    break
if not equal_neighbors:
    break

state = rng.choice(equal_neighbors)
consecutive += 1
sideways_moves += 1
visited.add(_freeze(state))
iterations += 1
history.append(objective(state, problem))
```

Urutan prioritas ini penting karena perbaikan selalu didahulukan atas sideways, sehingga sideways hanya aktif ketika benar-benar mentok. Karena skor tidak pernah turun, dan sideways berurutan dibatasi `max_sideways`, pencarian dijamin berhenti. Parameter `max_sideways = 0` membuat pengecekan langsung mem-break sebelum sideways pertama, sehingga perilakunya mirip dengan steepest ascent.

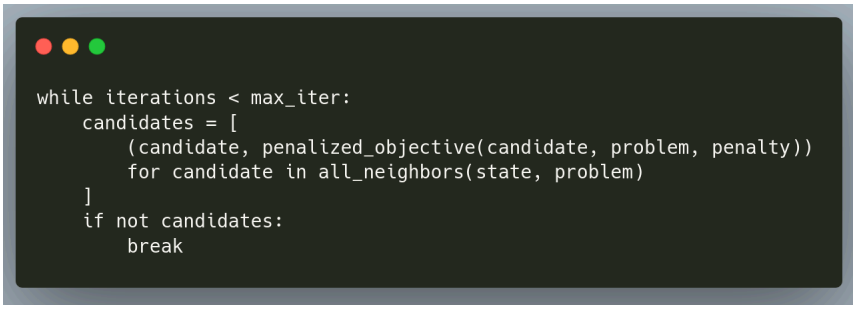
c. Stochastic Hill Climbing

Pada algoritma ini, pemilihan tetangga dilakukan secara acak di antara tetangga-tetangga yang memiliki nilai lebih baik dari state saat ini, bukan selalu memilih yang terbaik. Peluang pemilihan dapat dibuat proporsional terhadap besarnya peningkatan nilai, sehingga langkah yang lebih menjanjikan tetap lebih mungkin terpilih. Unsur random ini membuat jalur pencarian bervariasi dan dapat menghindari jebakan yang selalu dialami

oleh versi deterministik pada kondisi tertentu. Konvergensinya biasanya lebih lambat dibanding steepest ascent, namun pada kasus dengan output yang kompleks algoritma ini kadang menemukan solusi yang lebih baik.

1. Penyusunan kandidat

   Berbeda dari steepest yang memilih yang terbaik, stochastic membangun daftar yang meliputi `candidate` dan `score` dari seluruh neighbor, lalu mengundi satu elemen secara acak. Daftar ini dibuat ulang hanya setelah ada langkah yang diterima, sehingga selama fase stuck, undian diambil dari himpunan kandidat yang sama. Jika tidak ada kandidat sama sekali, loop langsung berhenti.

   ```
   while iterations < max_iter:
       candidates = [
           (candidate, penalized_objective(candidate, problem, penalty))
           for candidate in all_neighbors(state, problem)
       ]
       if not candidates:
           break
   ```

   Karena daftar dibangun dari `all_neighbors`, komposisinya didominasi relocate, yang jumlahnya jauh lebih banyak daripada swap dan rotate. Pemilihan dengan distribusi peluang yang seragam terhadap elemen daftar berarti peluang memilih relocate jauh lebih besar daripada peluang memilih swap atau rotate. Ini adalah konsekuensi desain yang membedakan stochastic di sini dari versi yang memilih jenis move terlebih dahulu.

2. Kriteria penerimaan neighbor

   Kandidat diterima hanya jika skornya lebih tinggi dari skor sekarang.

```
moved = False
while not moved:
    proposals += 1
    candidate, candidate_score = rng.choice(candidates)
    if candidate_score > score + EPSILON:
        state = candidate
        score = candidate_score
        stuck = 0
        iterations += 1
        history.append(objective(state, problem))
        moved = True
        continue
    stuck += 1
    if stuck >= max_stuck:
        break
if not moved:
    break
```

Penolakan menambah `stuck`, sedangkan penerimaan mereset `stuck`, menambah `iterations`, dan mencatat objective. proposals menghitung setiap pilihan, termasuk yang berulang pada kandidat yang sama, karena pemilihan dilakukan with replacement. `max_stuck = 1` mereproduksi perilaku klasik yang berhenti pada penolakan pertama.

Karena pemilihan berulang dari himpunan yang sama, kandidat yang sama bisa dipilih berkali-kali dan ditolak berkali-kali. proposals bisa jauh lebih besar daripada iterations. Jika `max_stuck` besar dan `p_t` kecil, algoritma bisa menghabiskan banyak pilihan sia-sia. Sebaliknya, jika `p_t` tinggi, hampir setiap pilihan diterima dan proposals ≈ iterations.

Jumlah pilihan sampai penerimaan pertama berdistribusi geometris dengan parameter p_t, sehingga:

$$E[proposals\ per\ langkah] = \frac{1}{p_t}.$$

Peluang berhenti setelah tepat `max_stuck` penolakan adalah:

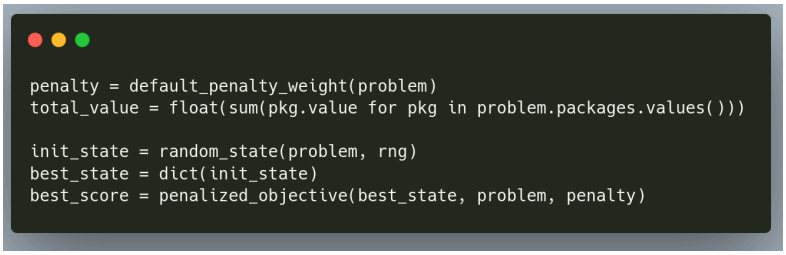
$$(1 - p_t)^{max_stuck}.$$

d. Random Restart Hill Climbing

Algoritma ini menjalankan hill climbing berulang kali, masing-masing dimulai dari state awal yang dibangkitkan secara acak, lalu menyimpan solusi terbaik dari semua percobaan. Karena setiap percobaan bersifat independen, kegagalan di satu maksimum lokal tidak memengaruhi percobaan berikutnya, dan peluang menemukan optimum global naik seiring bertambahnya jumlah restart.

Jika peluang sukses satu percobaan adalah *p*, maka rata-rata dibutuhkan sekitar $\frac{1}{p}$ restart untuk berhasil. Secara teori algoritma ini lengkap (*complete*) karena pada akhirnya akan membangkitkan state awal yang berada di dekat solusi, meskipun waktu komputasinya bisa besar untuk masalah dengan banyak maksimum lokal.

1. Batas atas global dan state terbaik



```
penalty = default_penalty_weight(problem)
total_value = float(sum(pkg.value for pkg in problem.packages.values()))

init_state = random_state(problem, rng)
best_state = dict(init_state)
best_score = penalized_objective(best_state, problem, penalty)
```

`total_value` dijumlahkan dari value seluruh paket dan dipakai sebagai batas atas objective, karena tidak mungkin memasukkan value melebihi total yang tersedia. Best state dilacak terpisah dari climb yang sedang berjalan, dimulai dari state awal acak. Skor terbaik awal dihitung dari state awal itu, bukan dari hasil climb pertama.

Pemisahan `best_state` dan state penting karena restart berikutnya tidak boleh menimpa hasil terbaik hanya karena climb barunya lebih buruk. best_state disalin dengan `dict(state)` agar tidak terkena aliasing saat `_steepest_climb` bekerja pada salinan state berikutnya. `total_value` juga dipakai sebagai ambang early stop, sehingga pencarian berhenti begitu solusi sempurna ditemukan.

2. Loop restart menggunakan _steepest_climb

Climb pertama memakai `init_state`; climb berikutnya memakai state acak baru dari rng yang sama, sehingga seluruh

rangkaian restart tetap deterministik terhadap seed tunggal. `restart_boundaries` mencatat indeks awal segmen history baru sebelum `history.extend`, sehingga hasil bisa dipecah per restart saat dianalisis. `iterations_per_restart` menyimpan biaya tiap climb.

```
for climb in range(max_restarts + 1):
    if climb == 0:
        climb_init = init_state
    else:
        climb_init = random_state(problem, rng)
        restart_boundaries.append(len(history))

    state, climb_history, climb_iterations, score = _steepest_climb(
        dict(climb_init), problem, penalty, max_iter
    )
    history.extend(climb_history)
    iterations += climb_iterations
    iterations_per_restart.append(climb_iterations)
```

Untuk climb ke-*k*, s_k = `random_state(rng)` dengan hasil $C(s_k)$. Total iterasi adalah $I = \sum_{k=0}^{k} I_k$ dan $I_k = iterations$ dari algoritma `_steepest_climb`. Fungsi $g(C(s_k)) \geq g(s_k)$ tetap berlaku karena tiap climb hanya menerima perbaikan.

3. Seleksi hasil terbaik

Climb hanya menggantikan best jika skornya benar-benar lebih tinggi. Setelah itu, jika best sudah menyentuh total_value – ε, loop berhenti lebih awal karena tidak ada solusi yang bisa lebih baik. Jumlah restart dihitung dari panjang `iterations_per_restart` dikurangi satu, sehingga restart yang tidak sempat berjalan tidak ikut terhitung.

```
    if score > best_score:
        best_score = score
        best_state = dict(state)
    if best_score >= total_value - EPSILON:
        break

restarts = len(iterations_per_restart) - 1
```

`restarts` yang dilaporkan adalah jumlah climb tambahan, bukan jumlah climb total. Dengan max_restarts = 10, maksimum ada 11 climb, tetapi restarts paling besar 10. Early stop membuat nilai restarts bisa jauh lebih kecil dari batas, dan itu memang tujuannya: berhenti begitu optimum tercapai daripada membuang waktu mengulang climb yang tidak mungkin memperbaiki hasil.

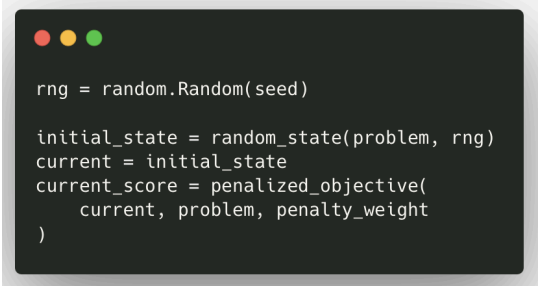
B. Simulated Annealing

Simulated Annealing mencari solusi dengan mencoba perubahan susunan paket secara acak. Perubahan yang meningkatkan skor selalu diterima, sedangkan perubahan yang menurunkan skor masih dapat diterima dengan peluang tertentu. Pendekatan ini membantu algoritma keluar dari optimum lokal. Peluang penerimaan dipengaruhi oleh temperatur yang terus diturunkan selama pencarian.

Implementasinya terdapat pada fungsi run() dalam file src/algorithms/simulated_annealing.py. Fungsi ini menerima data persoalan, *seed*, serta parameter pengendali pencarian. Nilai parameter bawaannya adalah temperatur awal (t0) sebesar 100, faktor pendinginan (alpha) sebesar 0,99982, temperatur minimum (t_min) sebesar 0,01, dan batas iterasi (max_iter) sebanyak 50.000. Penggunaan *seed* memungkinkan percobaan diulang dengan urutan keputusan acak yang sama.

1. Inisialisasi dan penilaian state

   Pencarian dimulai dari susunan acak yang dibentuk oleh random_state(). Susunan awal ini sudah memperhatikan batas ruang muat, tetapi belum tentu memenuhi batasan lainnya, seperti larangan tumpang tindih dan ketentuan penopangan paket. Oleh karena itu, penilaian state menggunakan total nilai paket yang dikurangi penalti pelanggaran.



```
rng = random.Random(seed)

initial_state = random_state(problem, rng)
current = initial_state
current_score = penalized_objective(
    current, problem, penalty_weight
)
```

   Fungsi penilaiannya dirumuskan sebagai berikut.

$$F(s) = f(s) - \lambda V(s).$$

Pada persamaan tersebut, $f(s)$ adalah total nilai paket yang dimuat, $V(s)$ adalah ukuran pelanggaran batasan, dan λ adalah bobot penalti. Dalam implementasi Simulated Annealing, bobot penalti bawaan menggunakan nilai paket terbesar dalam persoalan. Dengan penalti ini, penambahan paket belum tentu meningkatkan skor apabila penempatannya menimbulkan pelanggaran.

Selain state yang sedang ditelusuri, algoritma menyimpan state terbaik yang memenuhi seluruh batasan (*feasible*). State terbaik mula-mula berupa truk kosong dengan nilai nol. Jika state awal sudah *feasible*, state tersebut menjadi hasil terbaik sementara.

2. Pembangkitan dan penerimaan neighbor

Pada setiap iterasi, random_neighbor() menghasilkan satu kandidat melalui operasi *swap*, *relocate*, atau *rotate*. Skor kandidat kemudian dibandingkan dengan skor state saat ini.

```
candidate = random_neighbor(current, problem, rng)
candidate_score = penalized_objective(
    candidate, problem, penalty_weight
)
delta = candidate_score - current_score

if delta >= 0:
    accept = True
else:
    prob = math.exp(delta / temperature)
    accept = rng.random( ) < prob

if accept:
    current, current_score = candidate, candidate_score
```

Kandidat dengan skor lebih tinggi atau sama selalu diterima. Jika skornya lebih rendah, peluang penerimaan dihitung menggunakan persamaan berikut.

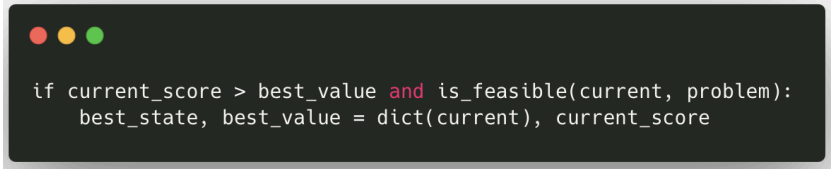
$$P(\text{diterima}) = \begin{cases} 1, & \Delta \geq 0, \\ \exp\left(\dfrac{\Delta}{T}\right), & \Delta < 0, \end{cases} \qquad \Delta = F(s') - F(s).$$

Nilai T menyatakan temperatur saat ini. Ketika temperatur masih tinggi, algoritma lebih terbuka terhadap penurunan skor sehingga dapat

menjelajahi susunan lain. Seiring penurunan temperatur, kandidat yang lebih buruk semakin sulit diterima. Pada temperatur yang sama, penurunan skor yang lebih besar juga menghasilkan peluang penerimaan yang lebih kecil.

3. Pembaruan hasil terbaik dan pendinginan

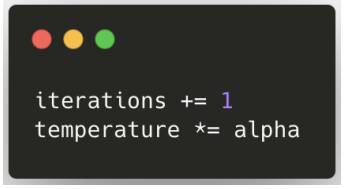
   Setelah keputusan penerimaan, algoritma memeriksa apakah state saat ini dapat menggantikan hasil terbaik. Pembaruan hanya dilakukan jika state tersebut *feasible* dan memiliki skor lebih tinggi.



```
if current_score > best_value and is_feasible(current, problem):
    best_state, best_value = dict(current), current_score
```

   Pada state *feasible*, penalti bernilai nol sehingga skor sama dengan total nilai paket yang dimuat. Dengan demikian, best_value menyatakan nilai objektif terbaik yang ditemukan tanpa melanggar batasan.

   Temperatur kemudian diturunkan menggunakan perkalian dengan faktor alpha pada setiap iterasi.



```
iterations += 1
temperature *= alpha
```

   Jadwal pendingan tersebut dapat dituliskan sebagai berikut.

$$T_{k+1} = \alpha T_k, \qquad 0 < \alpha < 1.$$

   Pencarian berlangsung selama temperatur masih lebih besar daripada t_min dan jumlah iterasi belum mencapai max_iter. Implementasi juga mencatat setiap periode 500 iterasi tanpa peningkatan hasil terbaik melalui parameter stuck_threshold. Pencatatan ini digunakan untuk analisis stagnasi dan tidak menghentikan pencarian.

4. Hasil pencarian

Fungsi run() mengembalikan objek SearchResult yang memuat susunan paket terbaik, nilai objektif, status kelayakan, jumlah iterasi, waktu eksekusi, dan riwayat pencarian.

Hasil akhirnya menggunakan best_state, yaitu state *feasible* terbaik yang ditemukan sepanjang pencarian. State ini dapat berbeda dari state pada iterasi terakhir karena algoritma masih mungkin menerima perpindahan yang menurunkan skor. Penyimpanan hasil terbaik memastikan bahwa perbaikan yang sudah ditemukan tetap dipertahankan.

Riwayat yang disimpan mencakup skor berpenalti state saat ini (history), nilai objektif terbaik yg *feasible*(best_feasible_history), temperatur, dan probabilitas penerimaan kandidat yang lebih buruk. Skor state saat ini dapat naik maupun turun, sedangkan nilai terbaik yang *feasible* tidak menurun. Data tersebut digunakan untuk membuat grafik dan menjelaskan perilaku algoritma pada bagian hasil dan analisis eksperimen.

C. Genetic Algorithm

Genetic Algorithm mencari solusi dengan mengembangkan sekumpulan kandidat yang disebut populasi. Setiap individu dalam populasi mewakili satu susunan paket. Pada setiap generasi, individu dipilih untuk menjadi induk (*parent*), kemudian dikombinasikan melalui *crossover* dan diubah melalui mutasi. Proses ini diulang untuk memperoleh susunan dengan total nilai paket yang lebih tinggi.

Implementasinya terdapat dalam file src/algorithms/genetic.py, dengan fungsi utama run(). Secara bawaan, algoritma menggunakan populasi sebanyak 100 individu, 500 generasi, peluang *crossover* sebesar 0,9, dan peluang mutasi sebesar 0,15. Pemilihan induk menggunakan turnamen berukuran tiga individu, sedangkan dua individu terbaik dipertahankan pada setiap generasi melalui *elitism*.

1. Representasi individu dan inisialisasi populasi

   Setiap individu menggunakan representasi State yang memetakan identitas paket ke posisi dan orientasinya. Paket yang berada di luar truk ditandai dengan None. Dengan demikian, satu individu menyimpan informasi pemilihan paket sekaligus susunannya di dalam truk.

   Populasi awal dibentuk menggunakan random_feasible_state(). Setiap paket memiliki peluang bawaan sebesar 0,8 untuk dicoba dimasukkan.

Paket hanya dimuat jika ditemukan penempatan yang memenuhi seluruh batasan.

```
rng = random.Random(seed)
population = [
    random_feasible_state(problem, rng, inclusion_probability)
    for _ in range(population_size)
]
fitnesses = _evaluate_population(population, problem)
```

Karena individu dalam populasi dijaga agar selalu *feasible,* nilai *fitness* langsung menggunakan total nilai paket yang dimuat, tanpa penalti.

```
def _evaluate_population(population, problem):
    return [
        objective(individual, problem)
        for individual in population
    ]
```

Individu terbaru pada populasi awal disimpan sebagai initial_state sekaligus hasil terbaik sementara. Penggunaan seed memungkinkan pembentukan populasi dan keputusan acak lainnya diulang secara konsisten.

2. Pemilihan induk

Fungsi tournament_selection() memilih sejumlah individu secara acak, kemudian mengambil individu dengan *fitness* tertinggi sebagai induk. Proses ini dilakukan dua kali untuk memperoleh pasangan induk.

```
candidate_indices = rng.sample(
    range(len(population)), tournament_size
)
winner_index = max(
    candidate_indices, key=fitnesses.__getitem__
)
return dict(population[winner_index])
```

Pemilihan ini memberikan kesempatan lebih besar kepada individu berkualitas tinggi untuk menghasilkan keturunan. Namun, individu lain tetap memiliki peluang untuk terpilih karena peserta turnamen ditentukan secara acak.

3. Crossover dan perbaikan susunan

   Penggabungan induk menggunakan *uniform crossover*. Untuk setiap paket, fungsi `uniform_crossover()` memilih informasi penempatan dari salah satu induk dengan peluang yang sama.

```
return {
    package_id: parent_a[package_id]
    if rng.random() < 0.5
    else parent_b[package_id]
    for package_id in parent_a
}
```

   Peluang 0.5 tersebut berlaku untuk pemilihan asal setiap paket, sedangkan `crossover_rate` menentukan apakah proses crossover dilakukan pada pasangan induk. Jika tidak dilakukan, children dibentuk dengan menyalin induk pertama.

```
if rng.random() < crossover_rate:
    child = uniform_crossover(parent_a, parent_b, rng)
else:
    child = dict(parent_a)

child = repair_state(
    child, problem, rng, fill_outside=False
)
```

   Walaupun kedua induk *feasible*, penggabungan susunannya dapat menimbulkan tumpang tindih atau pelanggaran lainnya. Fungsi `repair_state()` memperbaiki children dengan mempertahankan penempatan yang valid dari bawah ke atas, kemudian mencoba menempatkan ulang paket yang bermasalah. Paket yang tidak dapat ditempatkan dikembalikan ke luar truk.

Parameter `fill_outside=False` membuat proses perbaikan tidak otomatis mencoba memasukkan paket yang sejak awal berstatus None pada children. Hal ini membantu mempertahankan variasi pemilihan paket dalam populasi.

4. Mutasi dan pembentukan generasi berikutnya

   Setelah diperbaiki, children memiliki peluang untuk mengalami mutasi. Fungsi `mutate()` memilih secara acak salah satu operasi yang dapat diterapkan:

   - **Swap**, menukar informasi penempatan dua paket, termasuk paket di dalam dan di luar truk.
   - **Move**, mencoba memindahkan paket ke penempatan valid yang berbeda atau memasukkan paket dari luar truk.
   - **Rotate**, merotasi paket di dalam truk sebesar 90 derajat terhadap salah satu sumbu.

   Operasi *swap* dan *rotate* diikuti perbaikan susunan, sedangkan *move* mencari penempatan yang menghasilkan state *feasible*.

```
if rng.random() < mutation_rate:
    child = mutate(child, problem, rng)

next_population.append(child)
```

   Mutasi menambah variasi susunan yang tidak hanya berasal dari kombinasi kedua induk. Jika tidak ditemukan perubahan yang dapat diterapkan, individu dapat tetap sama. Sebelum menghasilkan children, algoritma menyalin individu terbaik dari generasi saat ini ke populasi berikutnya.

```
ranked_indices = sorted(
    range(population_size),
    key=fitnesses.__getitem__,
    reverse=True,
)
next_population = [
    dict(population[index])
    for index in ranked_indices[:elite_count]
]
```

Mekanisme *elitism* tersebut menjaga agar solusi terbaik tidak hilang akibat *crossover* atau mutasi. Sisa populasi diisi oleh anak hingga ukurannya kembali mencapai `population_size`.

5. Hasil pencarian

Setelah satu generasi terbentuk, seluruh individu dievaluasi kembali. Algoritma mencatat *fitness* maksimum dan rata-rata populasi, lalu memperbarui hasil terbaik apabila ditemukan peningkatan.

```
max_history.append(generation_best)
average_history.append(fmean(fitnesses))

if generation_best > best_fitness:
    best_fitness = generation_best
    best_state = dict(population[generation_best_index])
    best_iteration = iteration
```

Pencarian berhenti setelah jumlah generasi mencapai parameter `iterations`. Hasilnya dikembalikan melalui objek `SearchResult` yang berisi individu terbaik dari populasi awal, susunan terbaik selama pencarian, nilai objektif, waktu eksekusi, serta riwayat *fitness*.

Riwayat maksimum menunjukkan perkembangan solusi terbaik, sedangkan riwayat rata-rata menggambarkan kualitas populasi secara keseluruhan. Karena adanya *elitism*, *fitness* maksimum tidak menurun antargenerasi, sementara rata-rata populasi masih dapat naik maupun turun. Kedua riwayat ini digunakan untuk visualisasi dan analisis eksperimen.

II. Rancangan Eksperimen

A. Tujuan Eksperimen

Eksperimen dirancang untuk mengevaluasi kemampuan setiap algoritma dalam menghasilkan susunan paket yang feasible dan memiliki nilai objektif tinggi. Evaluasi tidak hanya memperhatikan nilai akhir, tetapi juga durasi pencarian, perkembangan nilai objektif selama pencarian, serta konsistensi hasil pada seed yang berbeda. Seluruh eksperimen wajib menggunakan objective `total_value` agar hasil Hill Climbing, Simulated Annealing, dan Genetic Algorithm dapat dibandingkan secara langsung pada persoalan yang sama.

B. Testcase Utama Eksperimen

Eksperimen menggunakan problem instance large.json. Testcase ini terdiri atas satu truk berukuran 14 x 24 x 8 dengan kapasitas berat maksimum 850 dan 45 paket yang mempunyai dimensi, value, weight, status fragile, serta ETA yang bervariasi. Total berat seluruh paket adalah 2.051, sedangkan total kapasitas truk hanya 850. Total volume seluruh paket adalah 5.421, sedangkan volume truk adalah 2.688. Oleh karena itu, seluruh paket tidak mungkin dimuat secara bersamaan dan algoritma harus menentukan kombinasi paket, posisi, serta orientasi yang memberikan total value terbesar. Daftar paket dibawah ini dipersingkat, file lengkap tersedia pada src/data/large.json di repository program kami.


```
{
  "name": "large",
  "truck": {
    "w": 14,
    "l": 24,
    "h": 8,
    "max_capacity": 850
  },
  "packages": [
    {
      "id": "L01",
      "w": 8,
      "l": 6,
      "h": 4,
      "value": 520,
      "weight": 75,
      "is_fragile": false,
      "eta": 2
```

```
    },
    {
      "id": "L02",
      "w": 10,
      "l": 5,
      "h": 3,
      "value": 410,
      "weight": 58,
      "is_fragile": false,
      "eta": 5
    }
  ]
}
```

C. Testcase Bonus Multi-Truk

Pengujian bonus menggunakan multi_truck.json. Testcase ini terdiri atas 18 paket dan tiga truk heterogen. Truk T01 berukuran 8 x 10 x 5 dengan kapasitas 170, T02 berukuran 7 x 9 x 5 dengan kapasitas 140, dan T03 berukuran 6 x 8 x 5 dengan kapasitas 120. Total kapasitas berat armada adalah 430, sedangkan total berat seluruh paket adalah 405. Testcase ini digunakan untuk memeriksa bahwa assignment paket, batas geometri, penopangan, larangan fragile sebagai support, dan kapasitas dihitung secara terpisah untuk setiap truk. Objective akhirnya tetap berupa total value seluruh paket yang dimuat oleh armada.

Perbedaan utama format multi-truk adalah penggunaan array trucks. Setiap truk memiliki identifier agar placement paket dapat menyimpan truk tujuan. Cuplikan dibawah ini merupakan format inputnya. File lengkap tersedia pada src/data/multi_truck.json di repository.

```
{
  "name": "multi_truck",
  "trucks": [
    {"id": "T01", "w": 8, "l": 10, "h": 5, "max_capacity": 170},
    {"id": "T02", "w": 7, "l": 9, "h": 5, "max_capacity": 140},
    {"id": "T03", "w": 6, "l": 8, "h": 5, "max_capacity": 120}
  ],
  "packages": [
    {
      "id": "P01",
      "w": 5,
      "l": 4,
      "h": 3,
```

```
    "value": 120,
    "weight": 24,
    "is_fragile": false,
    "eta": 3
   }
  ]
}
```


D. Konfigurasi Eksperimen

Seluruh algoritma dijalankan pada large.json dengan tiga seed, yaitu 0, 1, dan 2. Penggunaan seed yang sama memungkinkan hasil direproduksi sekaligus memberi variasi pada initial state dan keputusan acak selama pencarian.

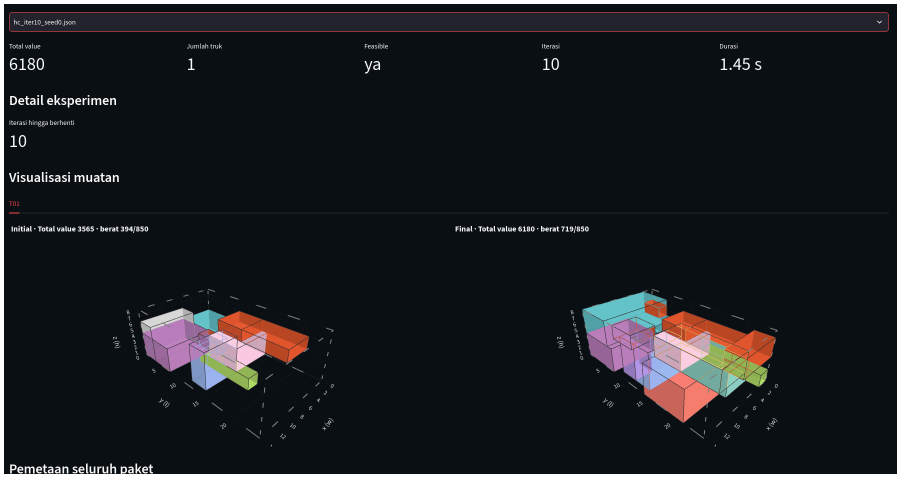
Steepest Ascent Hill Climbing dijalankan tiga kali dengan max_iter sebesar 500. Untuk setiap run dicatat initial state, final state, nilai objektif akhir, jumlah iterasi yang diterima hingga pencarian berhenti, durasi, feasibility, dan grafik nilai objektif terhadap iterasi.

Simulated Annealing dijalankan tiga kali menggunakan temperatur awal T0 = 100, faktor pendinginan alpha = 0,99982, temperatur minimum Tmin = 0,01, dan batas 50.000 iterasi. Bobot penalti pada large.json adalah 760, yaitu value terbesar dari sebuah paket pada testcase tersebut. `stuck_threshold` ditetapkan sebesar 500 iterasi. Konfigurasi yang sama digunakan pada ketiga seed agar perbedaan hasil berasal dari lintasan pencarian, bukan dari perubahan parameter.

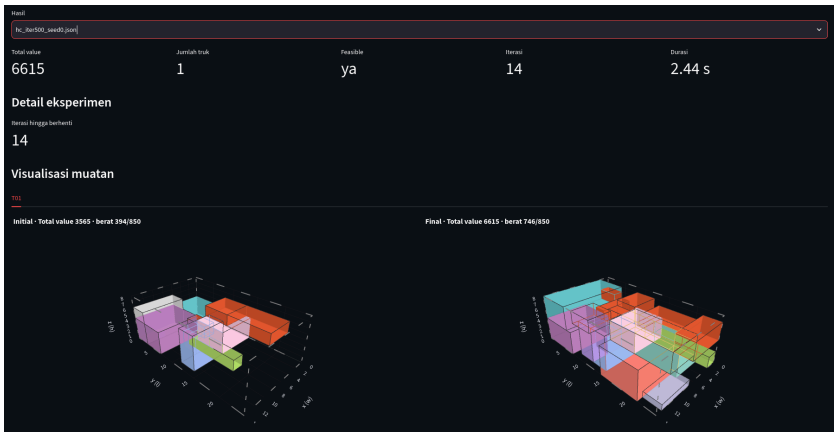
Genetic Algorithm diuji melalui dua kelompok eksperimen. Pada kelompok pertama, `population_size` dikontrol pada 100 individu dan iterations divariasikan menjadi 100, 500, dan 1.000. Pada kelompok kedua, iterations dikontrol pada 500 dan `population_size` divariasikan menjadi 25, 50, dan 100. Setiap konfigurasi dijalankan dengan seed 0, 1, dan 2. Dengan demikian, eksperimen GA menghasilkan 18 run.

E. Hasil Eksperimen Setiap Algoritma

1. Hill Climbing
    a. Steepest Ascent Hill Climbing
        - `Seed = 0, max_iter = 10`



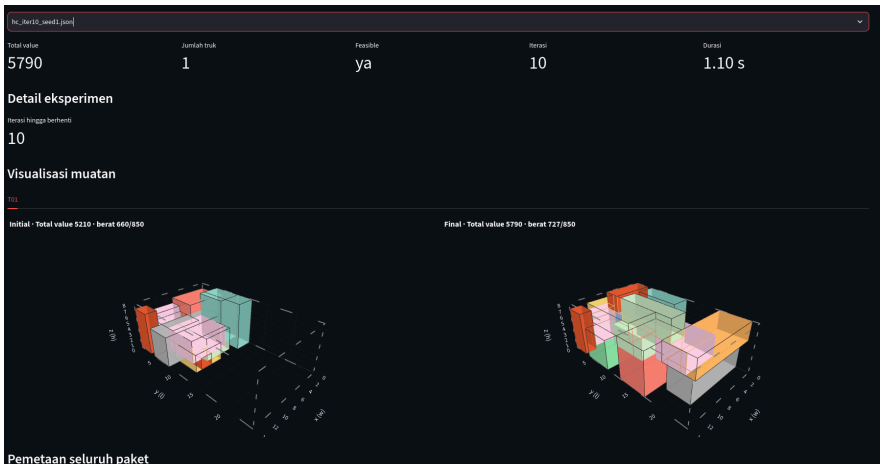
- Seed = 0, max_iter = 500



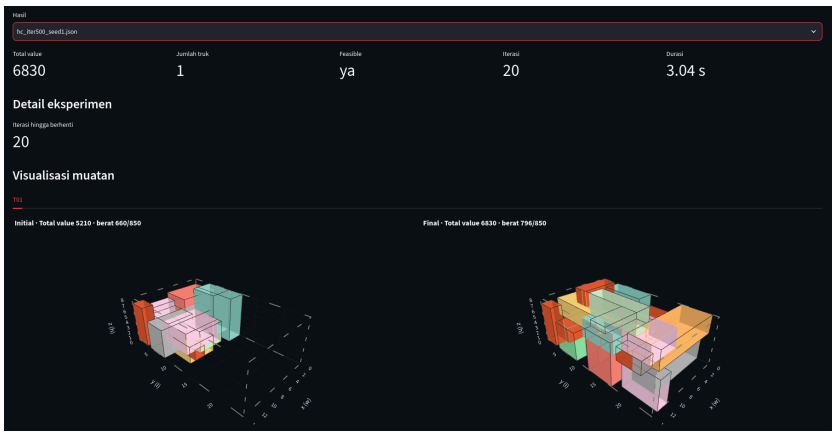
- Seed = 0, max_iter = 1000



- Seed = 1, max_iter = 10



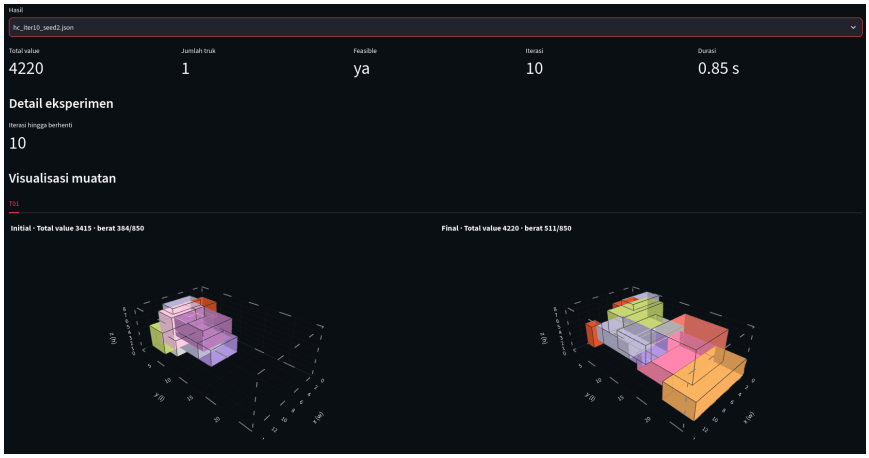
- Seed = 1, max_iter = 500



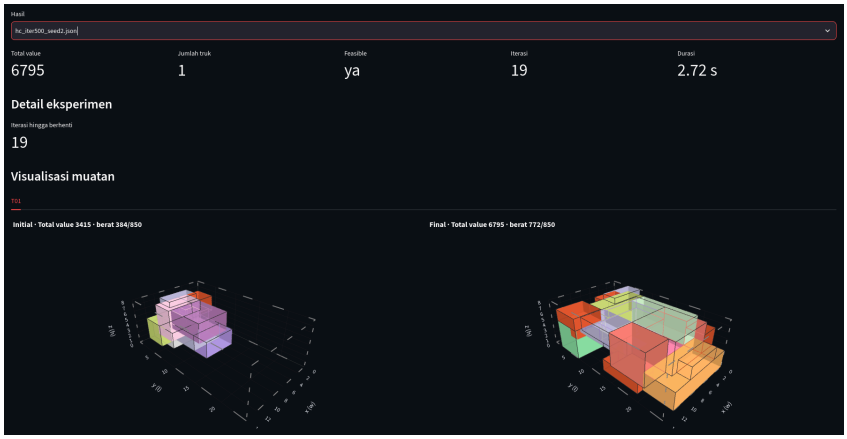
- Seed = 1, max_iter = 1000



- Seed = 2, max_iter = 10



- Seed = 2, max_iter = 500



- Seed = 2, max_iter = 1000

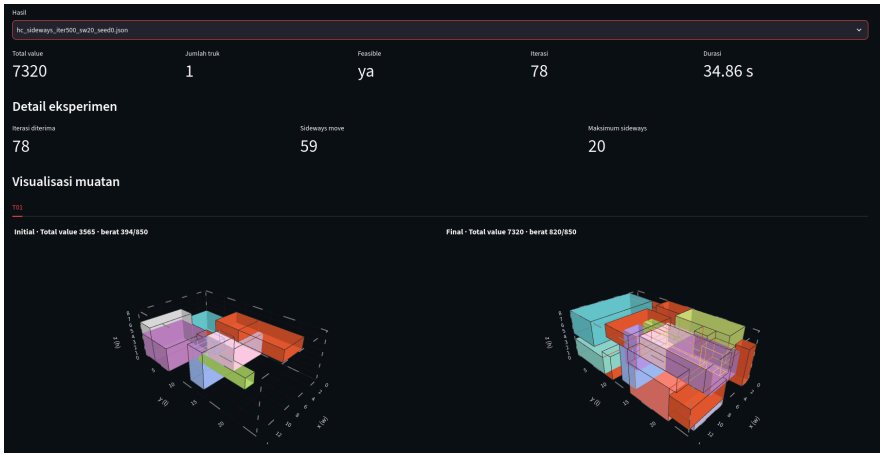


b. Sideways Move Hill Climbing

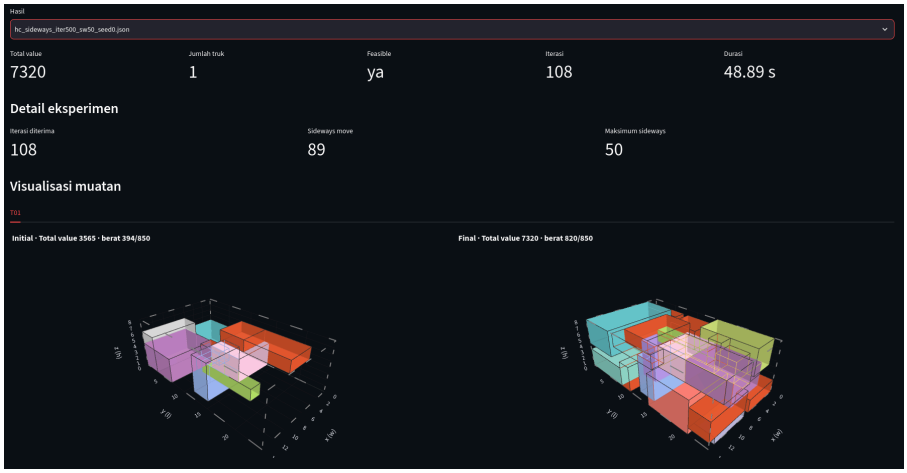
- Seed = 0, max_iter = 500, max_sideways = 5



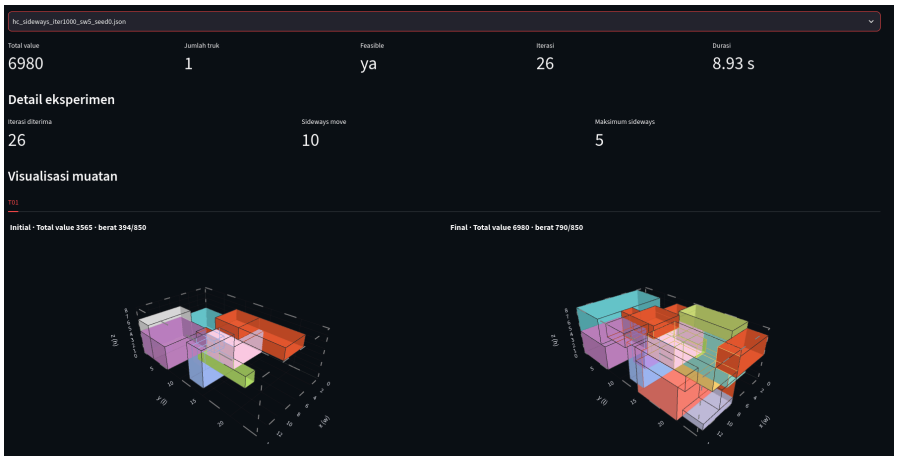
- Seed = 0, max_iter = 500, max_sideways = 20



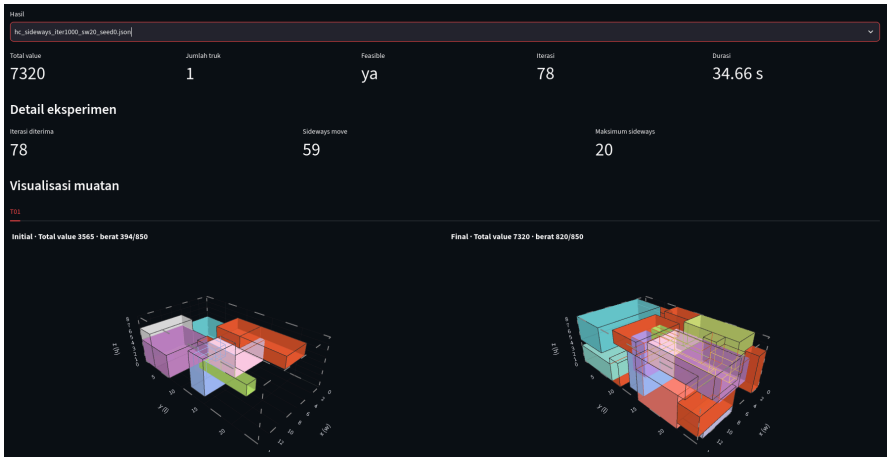
- Seed = 0, max_iter = 500, max_sideways = 50



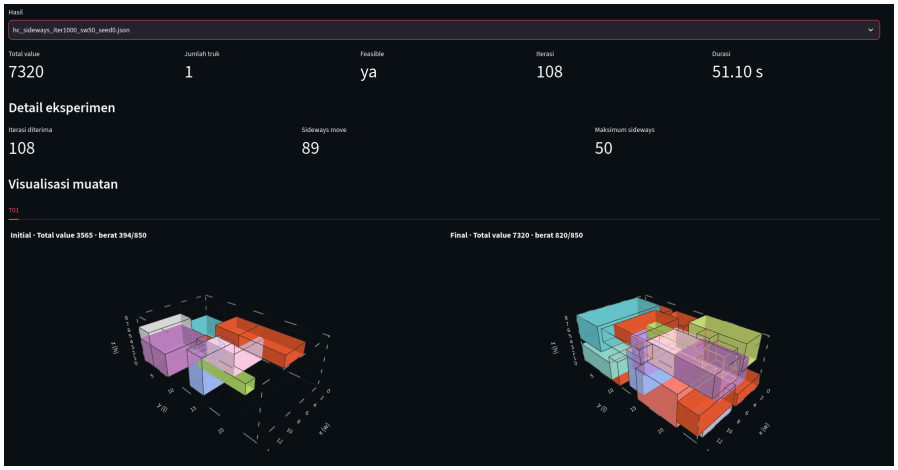
- Seed = 0, max_iter = 1000, max_sideways = 5



- Seed = 0, max_iter = 1000, max_sideways = 20

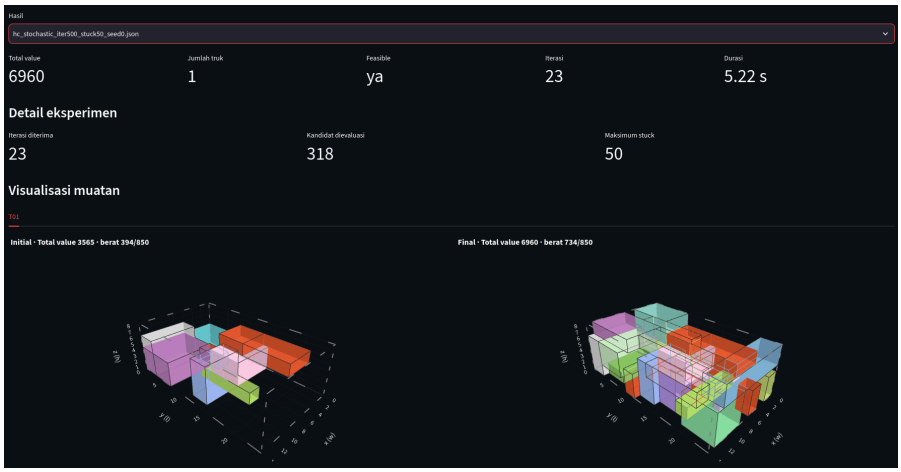


- Seed = 0, max_iter = 1000, max_sideways = 50

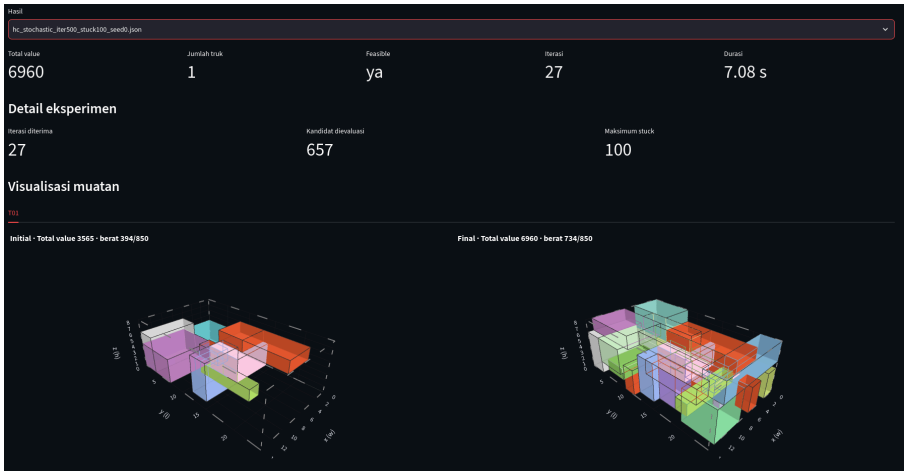


c. Stochastic Hill Climbing

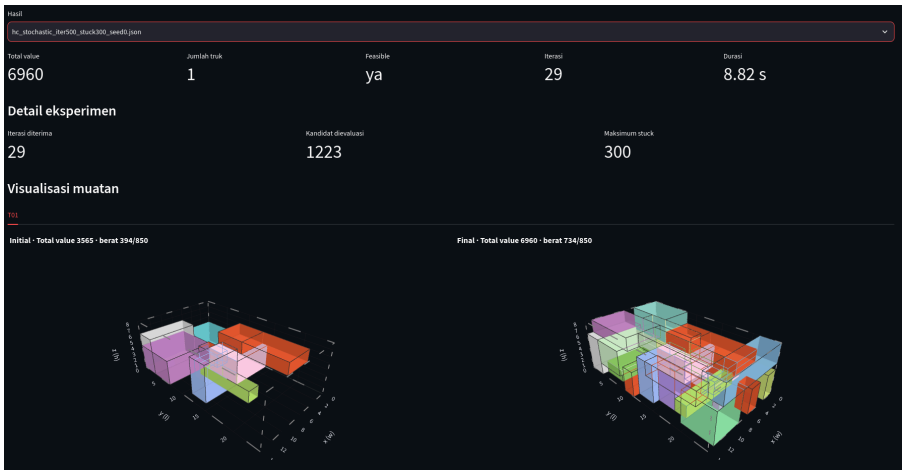
- `Seed = 0, max_iter = 500, max_stuck = 50`



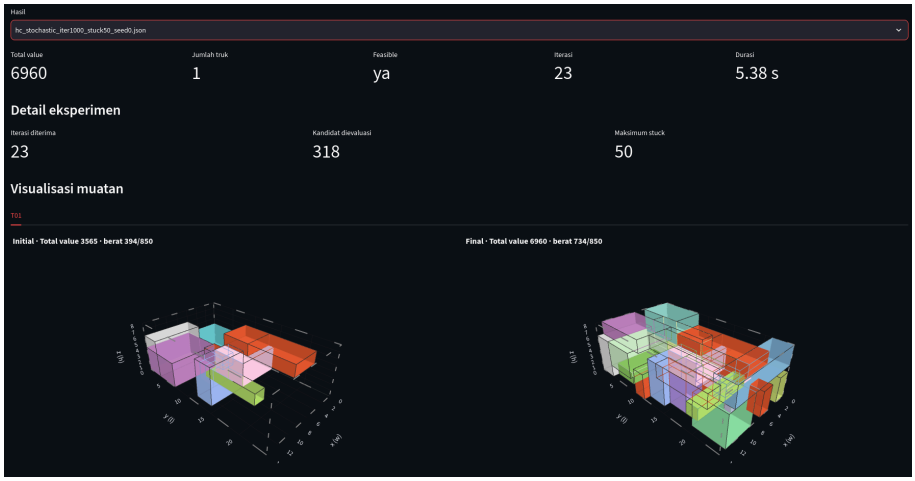
- `Seed = 0, max_iter = 500, max_stuck = 100`



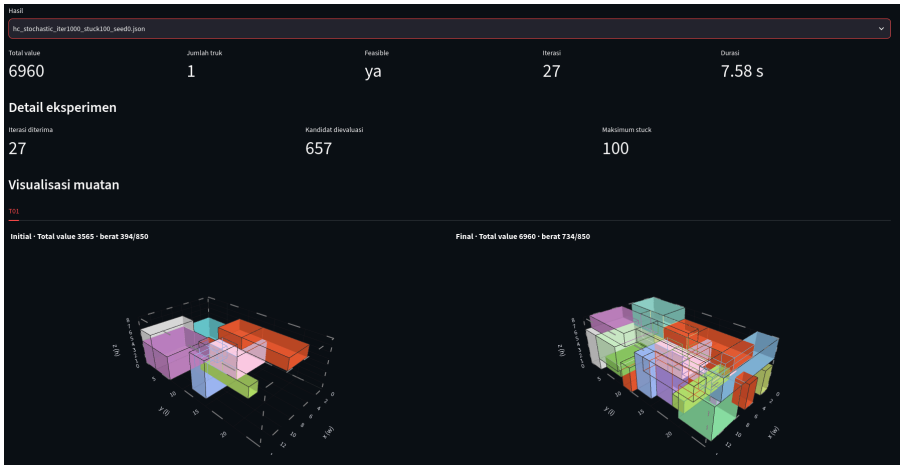
- Seed = 0, max_iter = 500, max_stuck = 300



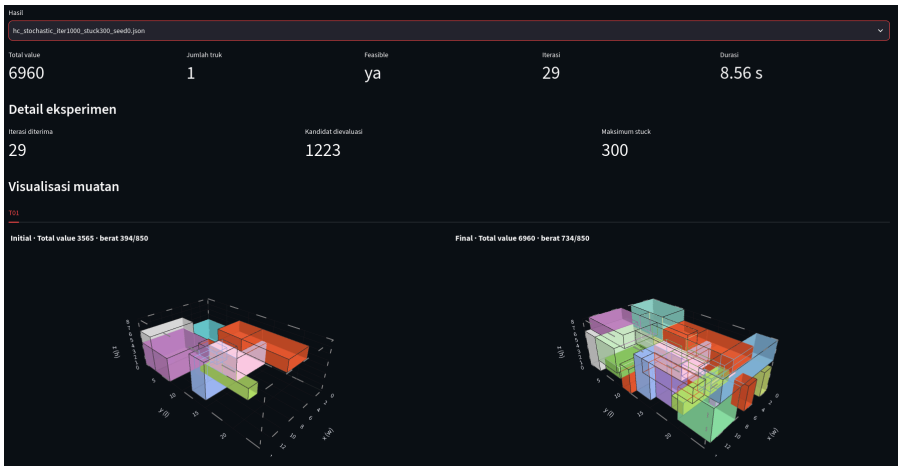
- Seed = 0, max_iter = 1000, max_stuck = 50



- Seed = 0, max_iter = 1000, max_stuck = 100



- Seed = 0, max_iter = 1000, max_stuck = 300

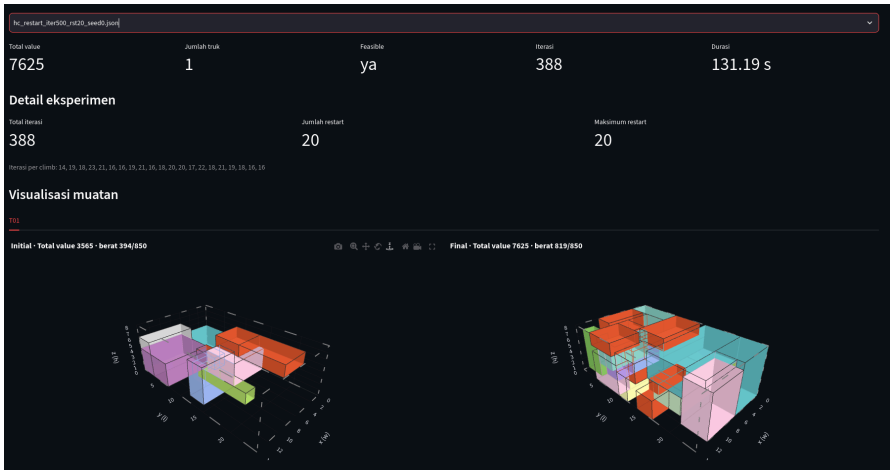


d. Random Restart Hill Climbing

- Seed =0, max_iter = 500, max_restart = 10



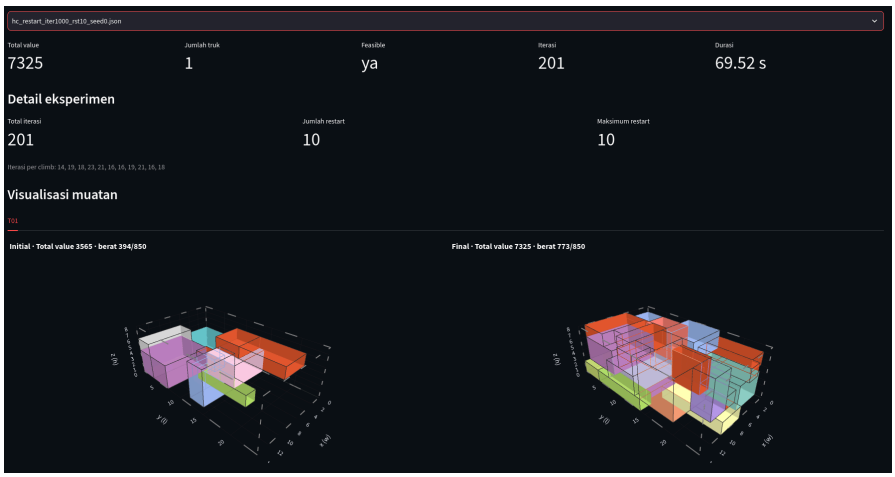
- Seed =0, max_iter = 500, max_restart = 20



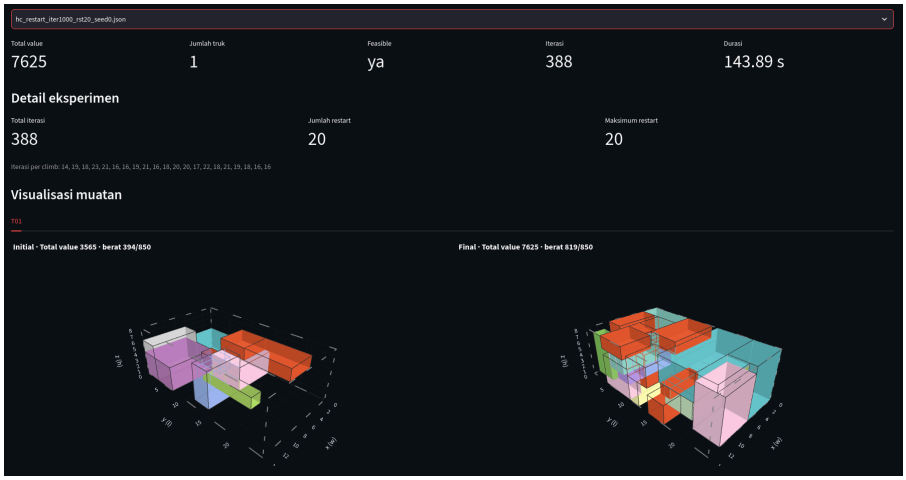
- Seed =0, max_iter = 500, max_restart = 50



- Seed =0, max_iter = 1000, max_restart = 10



- Seed =0, max_iter = 1000, max_restart = 20



- `Seed =0, max_iter = 1000, max_restart = 50`



2. Simulated Annealing

Pengujian dilakukan pada `large.json` menggunakan objective `total_value`. Algoritma dijalankan tiga kali dengan seed 0, 1, dan 2. Parameter yang digunakan adalah temperatur awal 100, alpha 0,99982, temperatur minimum 0,01, maksimum 50.000 iterasi, bobot penalti 760, dan stuck_threshold 500.

Tabel hasil pengujian Simulated Annealing

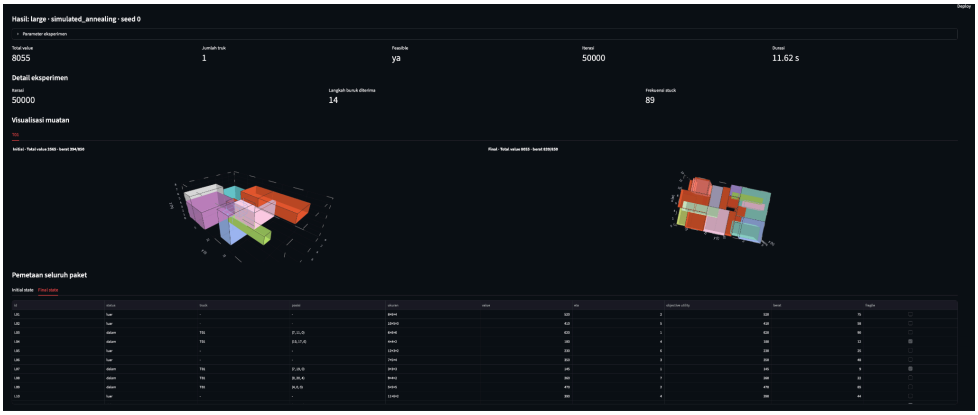
Percobaan	Seed	Objective akhir	Feasible	Durasi (detik)	Frekuensi stuck
1	0	8.055	Ya	11,62	89
2	1	7.320	Ya	11,66	90

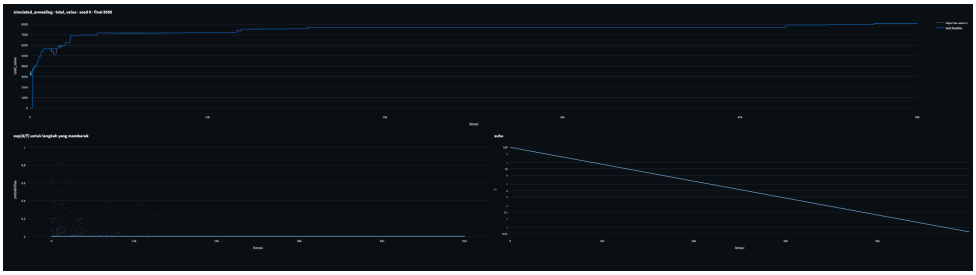
3	2	7.275	Ya	12,651	93

Rata-rata objective akhir adalah 7.550, dengan simpangan baku sampel 437,92. Rata-rata durasi pencarian adalah 11,975 detik. Seluruh percobaan menghasilkan solusi feasible dan berhenti setelah mencapai batas 50.000 iterasi.

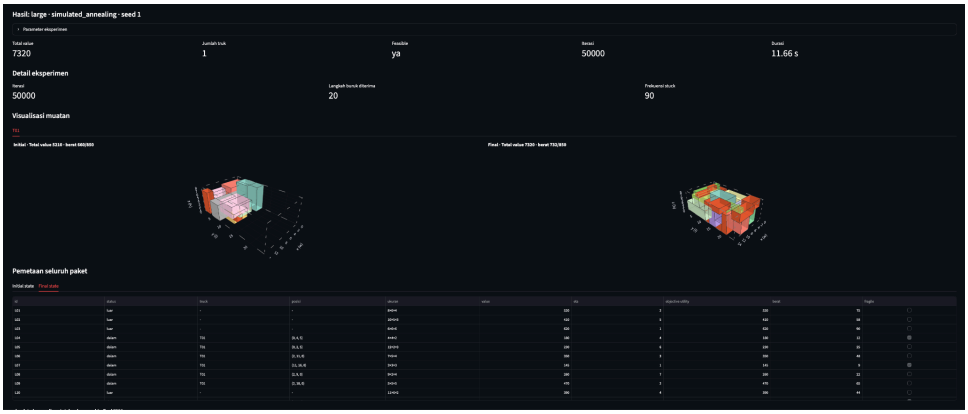
Dalam implementasi ini, satu kejadian *stuck* dicatat setiap kali terdapat 500 iterasi berturut-turut tanpa peningkatan solusi feasible terbaik. Pencatatan tersebut tidak menghentikan atau mengulang pencarian

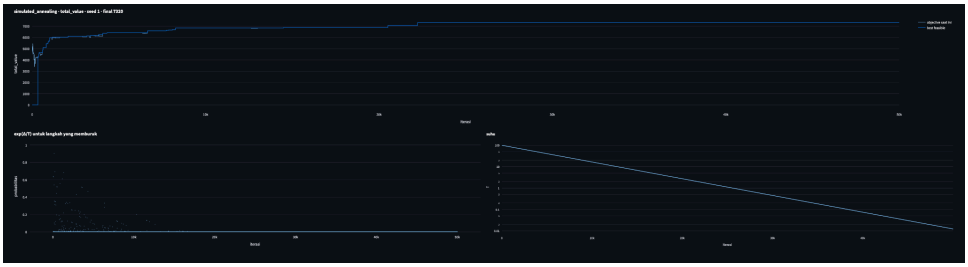
Percobaan 1 - Seed 0



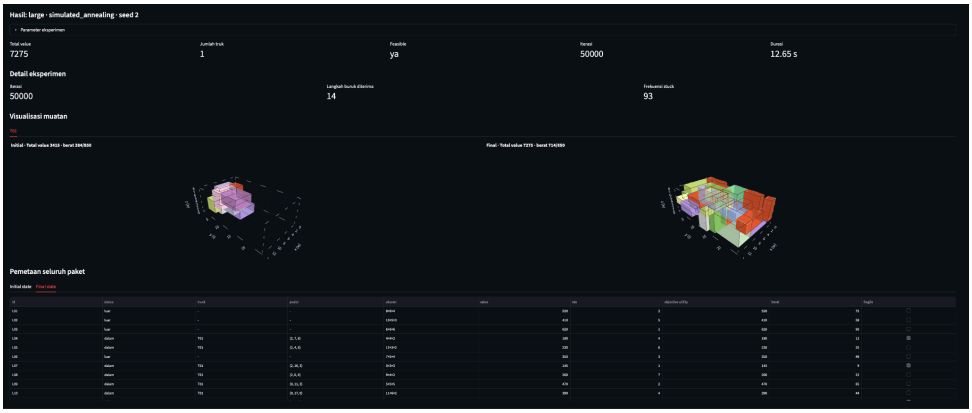


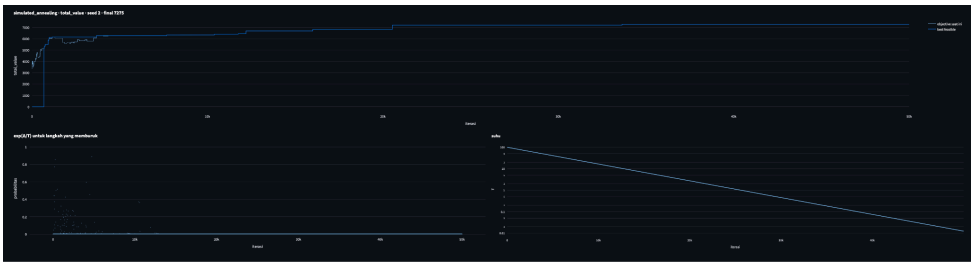
Percobaan 2 - Seed 1





Percobaan 3 - Seed 2





Percobaan pertama menghasilkan objective tertinggi, yaitu 8.055, sedangkan percobaan kedua dan ketiga menghasilkan 7.320 dan 7.275. Ketiganya memuat 26 paket, tetapi kombinasi paket yang dipilih berbeda

sehingga total nilainya juga berbeda. Grafik objective memperlihatkan peningkatan hasil yang cukup cepat pada tahap awal, kemudian perbaikan menjadi lebih jarang. Garis objective state saat ini dapat naik maupun turun karena algoritma memungkinkan penerimaan langkah yang memperburuk skor. Sementara itu, garis solusi feasible terbaik tidak menurun karena hasil terbaik selalu dipertahankan.

Grafik $e^{\Delta E/T}$ menampilkan nilai penerimaan untuk kandidat dengan skor lebih rendah, yaitu ketika $\Delta E < 0$. Pada tahap awal, beberapa kandidat tersebut masih memiliki peluang penerimaan yang cukup besar. Seiring penurunan temperatur, nilainya semakin mendekati nol. Kandidat dengan skor lebih baik atau sama diterima langsung. Frekuensi stuck tertinggi terdapat pada percobaan ketiga, yaitu 93 kejadian. Pada pengujian ini, percobaan tersebut juga menghasilkan objective akhir terendah. Namun, frekuensi stuck merupakan catatan periode tanpa perbaikan hasil terbaik, bukan bukti bahwa algoritma telah mencapai optimum lokal atau global.

3. Genetic Algorithm

Pada **studi pertama**, population_size dipertahankan sebesar 100 individu dan iterations divariasikan menjadi 100, 500, dan 1.000. Setiap konfigurasi dijalankan menggunakan seed 0, 1, dan 2.

Tabel Hasil eksperimen Genetic Algorithm dengan variasi banyak iterasi

Iterasi	Seed	Objective Akhir	Durasi (detik)	Generasi Terbaik
100	0	7.750	177,40	97
100	1	7.520	181,89	22
100	2	7.610	191,13	89
500	0	7.970	435,30	309
500	1	8.150	521,00	292
500	2	7.620	435,69	249
1000	0	7.970	745,53	309
1000	1	8.150	754,28	292

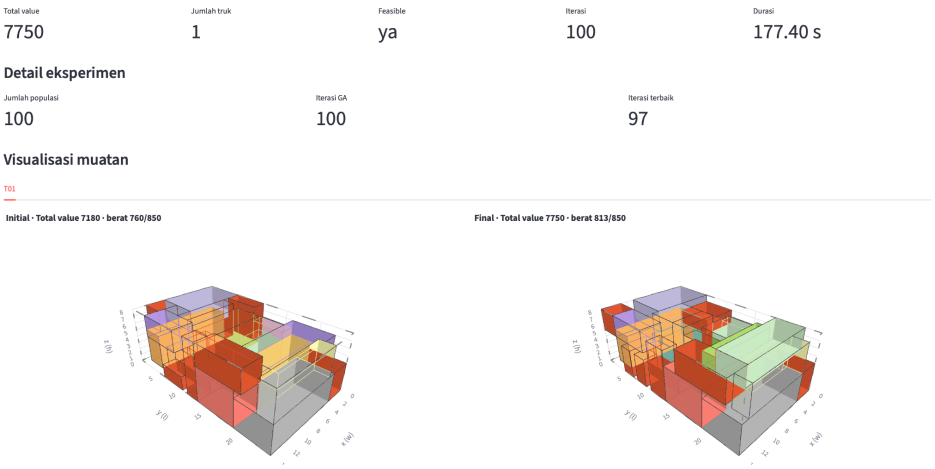
1000	2	7.670	485,94	618

Tabel Ringkasan eksperimen Genetic Algorithm dengan variasi banyak iterasi

Iterasi	Rata-rata Objective	Simpangan Baku	Minimum	Maksimum	Rata-rata Durasi (detik)
100	7.626,67	115,90	7.520	7.750	183,47
500	7.913,33	269,51	7.620	8.150	464,00
1000	7.930,00	242,49	7.670	8.150	661,92

Daftar Gambar:

- Hasil Genetic Algorithm dengan population_size 100, iterations 100, dan seed 0.



- Hasil Genetic Algorithm dengan population_size 100, iterations 100, dan seed 1.

Total value	Jumlah truk	Feasible	Iterasi	Durasi
7520	1	ya	100	181.89 s

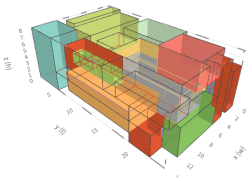
Detail eksperimen

Jumlah populasi	Iterasi GA	Iterasi terbaik
100	100	22

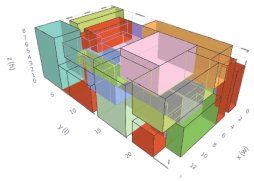
Visualisasi muatan

T01

Initial · Total value 7095 · berat 755/850



Final · Total value 7520 · berat 791/850



- Hasil Genetic Algorithm dengan population_size 100, iterations 100, dan seed 2.

Total value	Jumlah truk	Feasible	Iterasi	Durasi
7610	1	ya	100	191.13 s

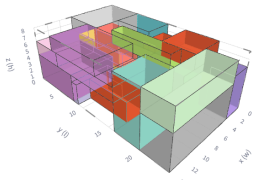
Detail eksperimen

Jumlah populasi	Iterasi GA	Iterasi terbaik
100	100	89

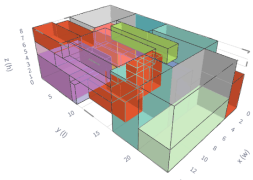
Visualisasi muatan

T01

Initial · Total value 7160 · berat 828/850



Final · Total value 7610 · berat 847/850



- Hasil Genetic Algorithm dengan population_size 100, iterations 500, dan seed 0.

Total value	Jumlah truk	Feasible	Iterasi	Durasi
7970	1	ya	500	435.30 s

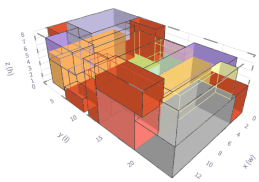
Detail eksperimen

Jumlah populasi	Iterasi GA	Iterasi terbaik
100	500	309

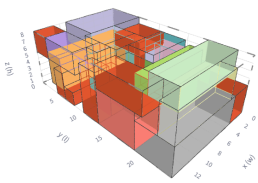
Visualisasi muatan

T01

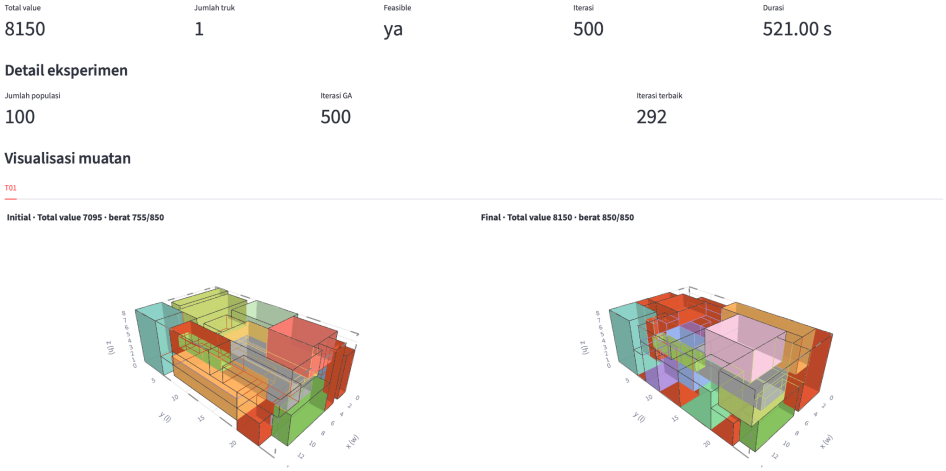
Initial · Total value 7180 · berat 760/850



Final · Total value 7970 · berat 846/850



- Hasil Genetic Algorithm dengan population_size 100, iterations 500, dan seed 1.



- Hasil Genetic Algorithm dengan population_size 100, iterations 500, dan seed 2.



- Hasil Genetic Algorithm dengan population_size 100, iterations 1.000, dan seed 0.

Total value	Jumlah truk	Feasible	Iterasi	Durasi
7970	1	ya	1000	745.53 s

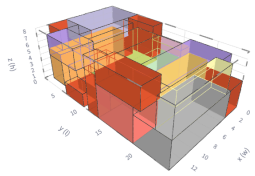
Detail eksperimen

Jumlah populasi	Iterasi GA	Iterasi terbaik
100	1000	309

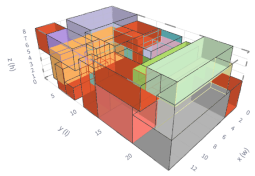
Visualisasi muatan

T01

Initial · Total value 7180 · berat 760/850



Final · Total value 7970 · berat 846/850



- Hasil Genetic Algorithm dengan population_size 100, iterations 1.000, dan seed 1.

Total value	Jumlah truk	Feasible	Iterasi	Durasi
8150	1	ya	1000	754.28 s

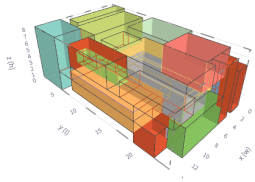
Detail eksperimen

Jumlah populasi	Iterasi GA	Iterasi terbaik
100	1000	292

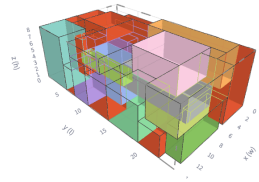
Visualisasi muatan

T01

Initial · Total value 7095 · berat 755/850



Final · Total value 8150 · berat 850/850



- Hasil Genetic Algorithm dengan population_size 100, iterations 1.000, dan seed 2.

Total value	Jumlah truk	Feasible	Iterasi	Durasi
7670	1	ya	1000	485.94 s

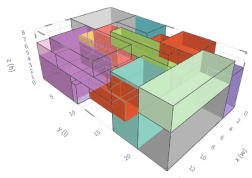
Detail eksperimen

Jumlah populasi	Iterasi GA	Iterasi terbaik
100	1000	618

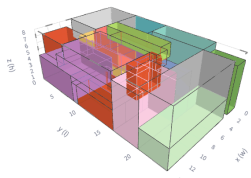
Visualisasi muatan

T01

Initial · Total value 7160 · berat 828/850



Final · Total value 7670 · berat 849/850



Pada **studi kedua**, iterations dipertahankan sebesar 500 dan `population_size` divariasikan menjadi 25, 50, dan 100. Setiap konfigurasi kembali dijalankan menggunakan seed 0, 1, dan 2.
Hasil eksperimen Genetic Algorithm dengan variasi jumlah populasi

Populasi	Seed	Objective Akhir	Durasi (detik)	Generasi Terbaik
25	0	7.615	57,57	83
25	1	7.660	60,25	298
25	2	7.610	62,87	180
50	0	7.895	131,82	280
50	1	7.770	125,59	459
50	2	7.690	126,21	496
100	0	7.970	282,54	309
100	1	8.150	338,19	292
100	2	7.620	281,02	249

Ringkasan eksperimen Genetic Algorithm dengan variasi jumlah populasi

Populasi	Rata-rata Objective	Simpangan Baku	Minimum	Maksimum	Rata-rata Durasi (detik)
25	7.628,33	27,54	7.610	7.660	60,23
50	7.785,00	103,32	7.690	7.895	127,87
100	7.913,33	269,51	7.620	8.150	300,85

- Hasil Genetic Algorithm dengan population_size 25, iterations 500, dan seed 0.

Total value	Jumlah truk	Feasible	Iterasi	Durasi
7615	1	ya	500	57.57 s

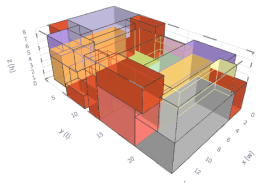
Detail eksperimen

Jumlah populasi	Iterasi GA	Iterasi terbaik
25	500	83

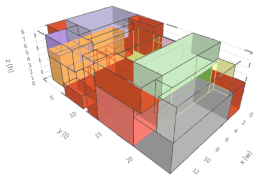
Visualisasi muatan

T01

Initial · Total value 7180 · berat 760/850



Final · Total value 7615 · berat 795/850



- Hasil Genetic Algorithm dengan population_size 25, iterations 500, dan seed 1.

Total value	Jumlah truk	Feasible	Iterasi	Durasi
7660	1	ya	500	60.25 s

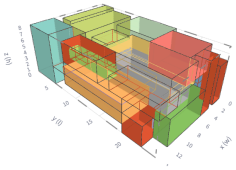
Detail eksperimen

Jumlah populasi	Iterasi GA	Iterasi terbaik
25	500	298

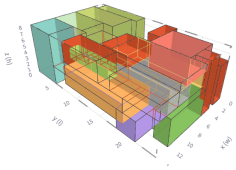
Visualisasi muatan

T01

Initial · Total value 7095 · berat 755/850



Final · Total value 7660 · berat 797/850



- Hasil Genetic Algorithm dengan population_size 25, iterations 500, dan seed 2.

Total value	Jumlah truk	Feasible	Iterasi	Durasi
7610	1	ya	500	62.87 s

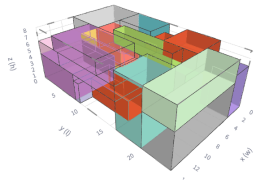
Detail eksperimen

Jumlah populasi	Iterasi GA	Iterasi terbaik
25	500	180

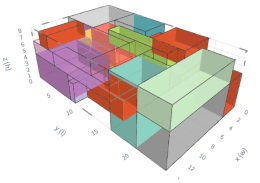
Visualisasi muatan

T01

Initial · Total value 7160 · berat 828/850

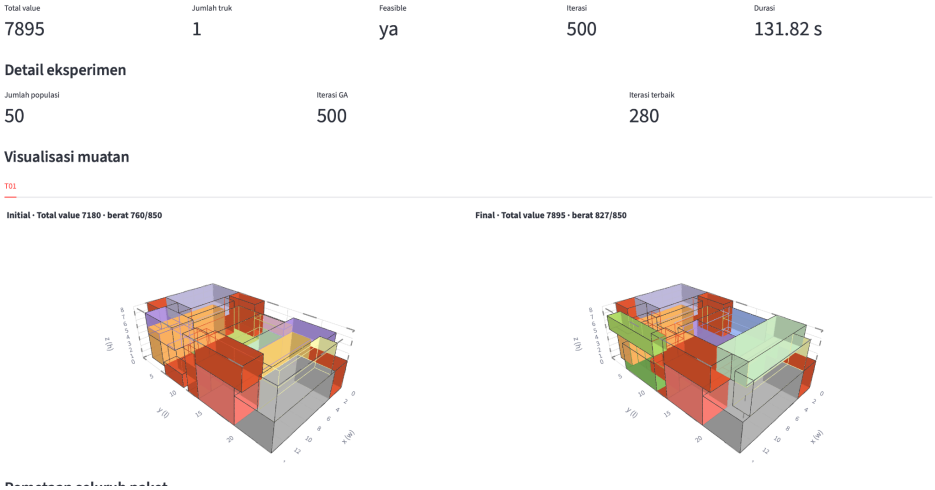


Final · Total value 7610 · berat 847/850

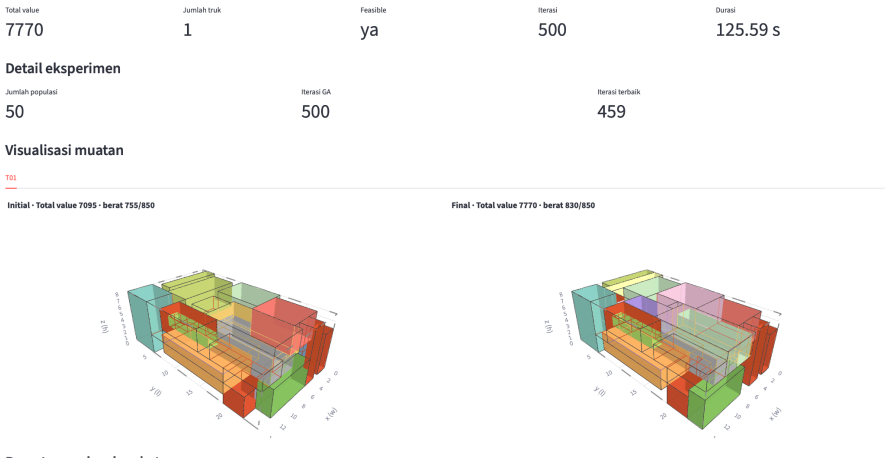


Pemetaan seluruh paket

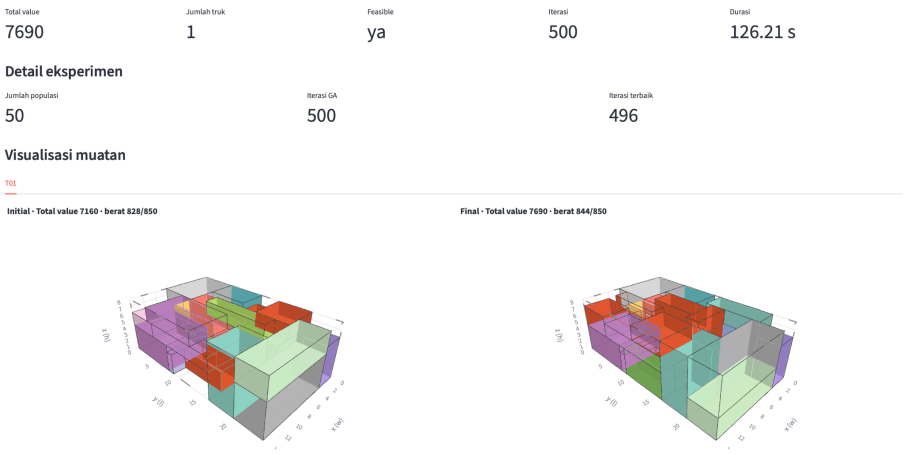
- Hasil Genetic Algorithm dengan population_size 50, iterations 500, dan seed 0.



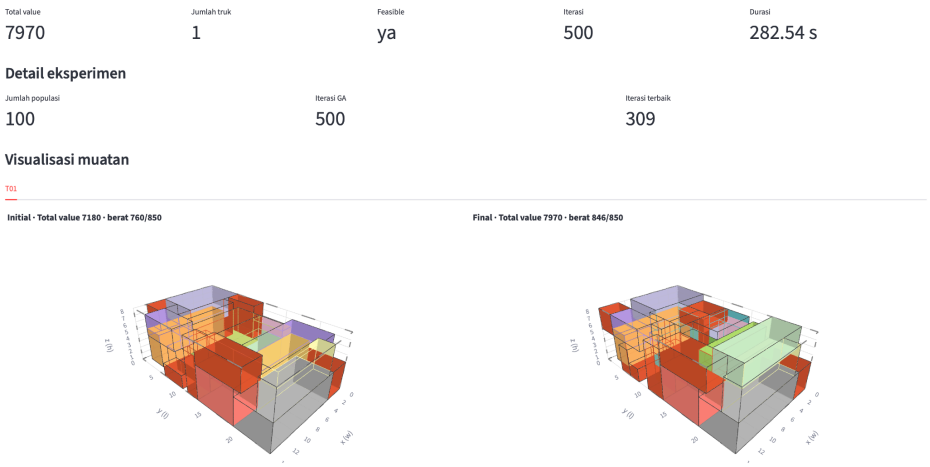
- Hasil Genetic Algorithm dengan population_size 50, iterations 500, dan seed 1.



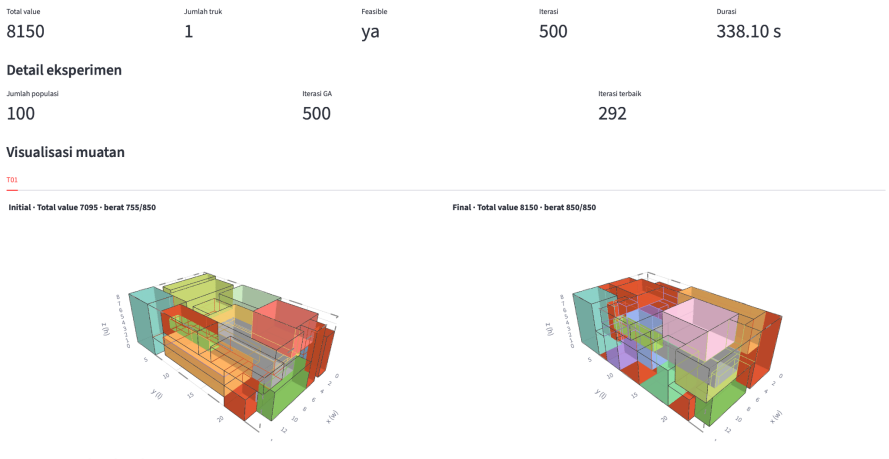
- Hasil Genetic Algorithm dengan population_size 50, iterations 500, dan seed 2.



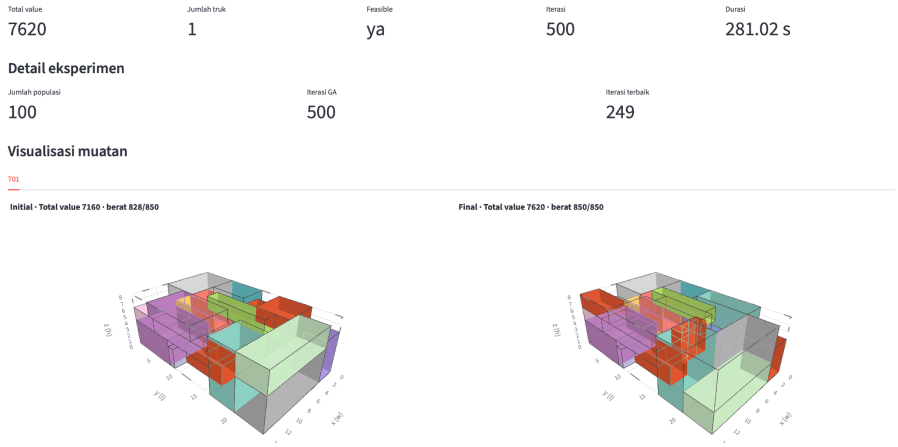
- Hasil Genetic Algorithm dengan population_size 100, iterations 500, dan seed 0.



- Hasil Genetic Algorithm dengan population_size 100, iterations 500, dan seed 1.



- Hasil Genetic Algorithm dengan population_size 100, iterations 500, dan seed 2.



F. Analisis Pengaruh Parameter

1. Hill Climbing dan variannya

Pada steepest ascent hill climbing, diperoleh bahwa perbedaan dalam max_iter hanya berpengaruh ketika max_iter lebih kecil dari jumlah iterasi jumlah maksimal diterima. Hal tersebut terjadi karena steepest ascent akan berhenti secara otomatis setelah mencapai optimum lokal, dimana pada kasus ini, iterasi yang dibutuhkan sangat kecil, hanya berjumlah sekitar belasan iterasi untuk mencapai optimum lokal. Hasil test case menggunakan max_iter = 10 menunjukkan penurunan skor karena algoritma berhenti sebelum berhasil mencapai optimum lokal. Pengaruh dari parameter seed tidak diukur secara pasti karena di-generate secara random.

Pada sideways move hill climbing, diperoleh bahwa perbedaan dalam max_iter tidak berpengaruh signifikan terhadap hasil output. Sedangkan, parameter max_sideways berpengaruh terhadap output, dimana terlihat bahwa penggunaan max_sideways = 20 menghasilkan skor yang lebih tinggi daripada max_sideways = 5. Namun, max_sideways = 50 menghasilkan skor yang sama dengan max_sideways = 20. Hal ini menunjukkan bahwa terdapat plateau pada kasus-kasus tersebut, namun tidak cukup sebanyak itu untuk memaksimalkan jumlah max_sideways yang telah diberikan

Pada stochastic hill climbing, diperoleh bahwa perubahan pada max_stuck dan max_iter tidak berpengaruh besar hasil kalkulasi.

max_stuck hanya akan berpengaruh apabila terdapat banyak plateau dan algoritma banyak mengalami stuck.

Untuk random restart hill climbing, diperoleh hasil terbesar dalam hill climbing pada max_restart = 20 dan max_restart = 50. max_restart berpengaruh terhadap jumlah pengulaman algoritma HC. Semakin besar max_restartnya, artinya algoritma akan lebih banyak menemukan solusi-solusi dengan skor yang lebih baik.

2. Simulated Annealing

Pada Simulated Annealing, temperatur awal dan faktor pendinginan mengatur peluang penerimaan langkah yang memperburuk skor. Pengujian menggunakan temperatur awal 100 dan alpha 0,99982. Pada tahap awal pencarian, temperatur masih cukup tinggi sehingga beberapa langkah yang memperburuk skor dapat diterima. Seiring penurunan temperatur, peluang tersebut semakin kecil, sebagaimana terlihat pada grafik probabilitas penerimaan. Mekanisme ini memberikan kesempatan untuk menjelajahi susunan lain sebelum pencarian menjadi lebih selektif.

Batas iterasi juga menentukan lamanya proses eksplorasi dan pendinginan. Ketiga percobaan berhenti setelah mencapai 50.000 iterasi. Pada saat tersebut, temperatur masih sekitar 0,01233, sedikit di atas temperatur minimum 0,01. Dengan demikian, kondisi berhenti yang tercapai pada pengujian ini adalah batas iterasi. Grafik objective menunjukkan bahwa perbaikan lebih sering terjadi pada tahap awal, sedangkan pada tahap berikutnya terdapat periode panjang tanpa peningkatan hasil terbaik.

Parameter `stuck_threshold` sebesar 500 digunakan untuk mencatat setiap periode 500 iterasi berturut-turut tanpa peningkatan solusi feasible terbaik. Frekuensi yang diperoleh adalah 89, 90, dan 93 kejadian. Parameter ini berfungsi sebagai pencatatan kondisi pencarian dan tidak memicu penghentian, restart, atau perubahan temperatur. Oleh karena itu, frekuensi stuck perlu dibaca bersama nilai threshold yang digunakan.

Seluruh parameter tersebut dipertahankan pada ketiga percobaan, sementara seed diubah menjadi 0, 1, dan 2. Perbedaan objective akhir sebesar 8.055, 7.320, dan 7.275 menunjukkan variasi hasil akibat state awal dan keputusan acak selama pencarian. Pengujian ini belum membandingkan nilai parameter yang berbeda, sehingga belum dapat menentukan apakah temperatur awal, alpha, atau batas iterasi yang

digunakan merupakan konfigurasi terbaik. Pembahasan pengaruh parameter didasarkan pada mekanisme implementasi dan perilaku pencarian yang teramati.

3. Genetic Algorithm

   Pada Genetic Algorithm, peningkatan iterations dari 100 menjadi 500 menaikkan rata-rata objective dari 7.626,67 menjadi 7.913,33. Namun, peningkatan hingga 1.000 iterasi hanya menghasilkan rata-rata 7.930,00 dengan durasi yang lebih lama. Hal ini menunjukkan bahwa penambahan iterasi setelah 500 memberikan peningkatan yang kecil dibandingkan dengan tambahan waktu komputasinya.

   Peningkatan population_size dari 25 menjadi 50 dan 100 juga menaikkan rata-rata objective dari 7.628,33 menjadi 7.785,00 dan 7.913,33. Populasi yang lebih besar memberikan keragaman individu dan kombinasi parent yang lebih tinggi sehingga peluang menemukan solusi berkualitas juga meningkat. Akan tetapi, durasi rata-rata ikut meningkat dari 60,23 detik pada populasi 25 menjadi 300,55 detik pada populasi 100.

   Perbedaan hasil antar seed menunjukkan bahwa GA masih dipengaruhi oleh proses acak pada pembentukan populasi awal, selection, crossover, dan mutation. Pada konfigurasi `population_size` = 100 dan iterations = 500, solusi terbaik sebesar 8.150 ditemukan pada seed 1 di generasi ke-292. Setelah mendekati solusi terbaik, peningkatan fitness mulai melambat karena populasi telah mengalami konvergensi.

   Berdasarkan hasil tersebut, `population_size` = 100 dan iterations = 500 dipilih sebagai konfigurasi yang paling seimbang antara kualitas solusi dan waktu eksekusi. Penambahan menjadi 1.000 iterasi membutuhkan waktu lebih lama, tetapi hanya memberi peningkatan rata-rata yang kecil. Nilai terbaik 8.150 juga telah mencapai 96,34% dari upper bound kapasitas sebesar 8.460, meskipun belum dapat dinyatakan sebagai global optimum karena masih terdapat constraint geometri.

G. Perbandingan Kinerja Antar Algoritma

Perbandingan dilakukan pada `large.json` menggunakan objective `total_value`. Pengujian utama membandingkan Steepest Ascent Hill Climbing dengan maksimum 500 iterasi, Simulated Annealing dengan maksimum 50.000 iterasi, serta Genetic Algorithm dengan populasi 100 dan 500 generasi. Setiap

konfigurasi menggunakan seed 0, 1, dan 2. Data GA diambil dari studi variasi jumlah iterasi agar sumber hasil dan durasinya konsisten.

Tabel perbandingan kinerja algoritma utama

Algo	Rata-rata objective	Simpangan baku	Min	Max	Avg durasi (s)	Feasible
Steepest Ascent HC	6.746,67	115,36	6.615	6.830	2,734	3/3
Simulated Annealing	7.550,00	437,92	7.275	8.055	11,975	3/3
Genetic Algorithm	7.913,33	269,51	7.620	8.150	463,996	3/3

Dari sisi kualitas solusi, GA menghasilkan rata-rata objective tertinggi, yaitu 7.913,33, serta hasil terbaik sebesar 8.150. Rata-ratanya sekitar 4,81% lebih tinggi daripada SA. Namun, peningkatan kualitas tersebut disertai waktu eksekusi yang jauh lebih lama. Pada konfigurasi yang dibandingkan, GA membutuhkan rata-rata sekitar 464 detik, sedangkan SA membutuhkan sekitar 12 detik.

Steepest Ascent HC memiliki durasi paling singkat, yaitu rata-rata 2,734 detik. Algoritma ini juga menghasilkan simpangan baku terkecil, sehingga nilai akhirnya relatif konsisten pada ketiga seed. Meskipun demikian, seluruh hasilnya masih lebih rendah daripada hasil SA dan GA. Konsistensi tersebut menunjukkan kestabilan hasil pada pengujian ini, tetapi belum menunjukkan kualitas solusi yang paling baik.

SA memberikan hasil di antara HC dan GA, baik dari rata-rata objective maupun durasinya. Dibandingkan HC, rata-rata objective SA meningkat sekitar 11,91%, dengan waktu pencarian yang lebih panjang. SA juga menghasilkan objective 8.055 pada seed 0, melampaui hasil GA sebesar 7.970 pada seed yang sama. Akan tetapi, simpangan baku SA paling besar di antara ketiga algoritma, yang menunjukkan bahwa kualitas hasilnya lebih bervariasi antarseed.

Varian tambahan HC dibandingkan secara terpisah karena hasil yang tersedia hanya menggunakan seed 0. Tabel berikut mengambil konfigurasi yang dibahas pada analisis parameter, dengan max_iter sebesar 500.

Varian	Parameter tambahan	Objective akhir	Durasi (detik)
HC Sideways Move	Max_sideways = 20	7.320	34,860
Stochastic HC	Max_stuck = 100	6.960	7,083
Random Restart HC	Max_restarts = 20	7.625	131,193

Ketiga varian menghasilkan objective lebih tinggi daripada Steepest Ascent HC pada seed 0, yaitu 6.615. Random Restart HC memberikan hasil tertinggi di antara varian HC tersebut, tetapi membutuhkan waktu paling lama. Peningkatan batas restart menjadi 50 tetap menghasilkan objective 7.625 dengan durasi lebih panjang, sehingga tambahan pencarian belum memberikan perbaikan pada percobaan tersebut.

Pada seed yang sama, SA menghasilkan objective 8.055 dalam 11,616 detik. Hasil ini lebih tinggi daripada seluruh varian HC yang dibandingkan, dengan durasi lebih singkat daripada Sideways Move dan Random Restart. Namun, hasil satu seed belum cukup untuk menyimpulkan konsistensi keunggulan tersebut pada percobaan lain.

Secara keseluruhan, pada dataset dan konfigurasi yang diuji, Steepest Ascent HC unggul dalam kecepatan, GA menghasilkan kualitas solusi tertinggi, dan SA memberikan keseimbangan antara kualitas solusi dan waktu eksekusi. Jumlah iterasi tidak digunakan sebagai ukuran biaya komputasi yang setara karena satu iterasi HC, satu iterasi SA, dan satu generasi GA memiliki pekerjaan yang berbeda. Perbandingan waktu juga menggunakan durasi yang tercatat pada masing-masing eksperimen, sehingga kesimpulannya terbatas pada pengukuran tersebut.

III. Pengembangan Fitur Bonus

A. Objective Function Tambahan

Selain total_value, program menyediakan objective priority_value yang mempertimbangkan nilai paket dan urgensi pengiriman berdasarkan ETA. Paket dengan ETA lebih kecil memperoleh pengali nilai yang lebih besar. Objective ini menggunakan atribut value dan eta yang sudah tersedia, sehingga tidak membutuhkan atribut input tambahan.

$$f_{\text{priority}}(s) = \sum_{i=1}^{n} a_i v_i \left(1 + \frac{\eta_{\max} - \eta_i}{\max(1, \eta_{\max})}\right).$$

Pada persamaan tersebut, a_i bernilai 1 apabila paket dimuat dan 0 apabila tidak dimuat, v_i merupakan nilai paket, η_i merupakan ETA paket, dan $\eta_{\max}$ merupakan ETA terbesar pada dataset. Penyebut dibatasi minimal 1 agar perhitungan tetap terdefinisi ketika seluruh ETA bernilai nol.

Sebagai contoh, pada large.json, ETA terbesar adalah 7. Paket L01 mempunyai nilai 520 dan ETA 2, sehingga kontribusinya terhadap objective prioritas menjadi:

$$520 \left(1 + \frac{7 - 2}{7}\right) \approx 891{,}43.$$

Implementasinya terdapat pada fungsi _build_priority_value() dalam src/core/objectives.py. Fungsi resolve_objective() memilih evaluator sesuai nama objective, kemudian evaluator tersebut digunakan oleh algoritma pencarian. Objective dapat dipilih melalui input Objective pada interface.

Pengujian utama dalam laporan tetap menggunakan total_value. Untuk priority_value, tes otomatis memeriksa perhitungan utility, pengaruh ETA, serta integrasinya dengan algoritma pencarian. Hasil eksperimen pencarian menggunakan objective ini belum disajikan dalam laporan.

B. Implementasi Varian Hill Climbing

Program mengimplementasikan Steepest Ascent Hill Climbing, Stochastic Hill Climbing, Hill Climbing dengan Sideways Move, dan Random Restart Hill Climbing.

Stochastic Hill Climbing menggunakan pemilihan neighbor secara acak. Sideways Move memungkinkan perpindahan ke neighbor dengan skor yang sama untuk menelusuri plateau. Random Restart menjalankan pencarian kembali dari state awal yang berbeda.

Penjelasan fungsi, potongan source code, dan hasil pengujian masing-masing varian telah disajikan pada bagian Implementasi Algoritma Local Search, Hill Climbing dan Variannya, serta bagian hasil eksperimen dan analisis parameter.

C. Dukungan Multi-Truk

Program mendukung penempatan paket pada lebih dari satu truk, baik truk identik maupun truk dengan dimensi dan kapasitas berbeda. Pada persoalan multi-truk, penempatan paket menyimpan identifier truk selain posisi dan orientasinya:

```
(truck_id, x, y, z, orientation)
```

Setiap paket memiliki satu penempatan atau berada di luar armada. Batas dimensi, kapasitas berat, overlap, penopangan, dan aturan paket rapuh diperiksa pada masing-masing truk. Objective total_value menghitung jumlah nilai seluruh paket yang dimuat oleh armada.

Implementasinya menggunakan model armada pada src/core/models.py dan fungsi pendukung pada src/core/fleet.py. Visualisasi menyediakan tab untuk setiap truk agar pembagian paket dapat diperiksa.

KESIMPULAN & SARAN

I. Kesimpulan

Ketiga kelompok algoritma berhasil menghasilkan susunan paket yang memenuhi constraint kapasitas, dimensi, overlap, support, dan fragile. Hill Climbing memiliki waktu eksekusi paling singkat, tetapi lebih mudah terjebak pada optimum lokal. Random Restart menjadi varian Hill Climbing terbaik dengan objective 7.625, sedangkan Simulated Annealing memberikan kompromi yang baik antara kualitas solusi dan waktu dengan hasil terbaik 8.055.

Genetic Algorithm menghasilkan solusi terbaik secara keseluruhan dengan objective 8.150 pada `population_size` = 100, iterations = 500, dan seed = 1. Hasil tersebut mencapai 96,34% dari upper bound kapasitas sebesar 8.460. Namun, GA membutuhkan waktu komputasi paling lama. Oleh karena itu, pemilihan algoritma bergantung pada kebutuhan: Hill Climbing untuk kecepatan, Simulated Annealing untuk keseimbangan, dan Genetic Algorithm untuk kualitas solusi.

II. Saran

Eksperimen berikutnya sebaiknya menggunakan lebih banyak seed dan variasi dataset agar perbandingan lebih representatif. Parameter Simulated Annealing, seperti T0, alpha, dan Tmin, juga perlu divariasikan secara langsung. Pada Genetic Algorithm, pengujian dapat diperluas terhadap mutation rate, crossover rate, ukuran tournament, dan elitism untuk memperoleh konfigurasi yang lebih optimal.

Waktu eksekusi dapat dikurangi dengan mengoptimalkan proses pengecekan constraint dan repair solusi, misalnya melalui penyimpanan hasil pemeriksaan atau pencarian posisi yang lebih efisien. Penelitian selanjutnya juga dapat menggunakan metode hybrid, seperti menggabungkan Genetic Algorithm dengan Hill Climbing, serta membandingkan hasil terhadap solver eksak agar jarak solusi terhadap global optimum dapat diketahui dengan lebih pasti.

LAMPIRAN

I. Link Repository

https://github.com/ethj0r/IF3170_G06_JensensLeatherJackets.git

II. Pembagian Tugas

NIM	Pembagian Tugas
13524026	- Menyusun arsitektur awal program - Mengembangkan representasi state dan pembangkitan neighbors - Menyusun dan mengimplementasikan Simulated Annealing - Mengembangkan visualisasi program - Menyusun laporan
13524031	- Menyelesaikan algoritma Hill climbing - Menyelesaikan bonus varian hill climbing - Menyusun laporan bagian algoritma hill climbing
13524067	- Membuat rencana dan mengimplementasikan core algorithms - Membuat rencana dan mengimplementasikan menyelesaikan genetic algorithm - Membuat rencana dan mengimplementasikan bonus objective dan multi-truck - Menyusun laporan bagian algoritma genetic algorithm - Debugging

III. Referensi

Streamlit, “Streamlit Documentation.” Available: https://docs.streamlit.io/.

Plotly, “3D Mesh Plots in Python.” Available: https://plotly.com/python/3d-mesh/.

chadiik, “Freight-packer: Freight Packing Solver & Visualizer,” repositori GitHub. Available: https://github.com/chadiik/freight-packer.

IV. Statement Penggunaan AI

https://drive.google.com/file/d/1KMkaFTJT5lWaFirjFqCaJOs0DqghQhr7/view?usp=sharing
Dokumen PDF Surat Penggunaan AI